

Fast-Fourier Transform (FFT) and Number-Theoretic Transform (NTT) - FFT, NTT and LBC

The need to make systems and algorithms faster so as to make them more practical has been ever constant in the world of software engineering and cryptography at large, and lattice-based cryptography is not left out.

Number-theoretic Transform (NTT) was used in ML-KEM (Module-Lattice-Based Key-Encapsulation Mechanism), a lattice-based cryptographic algorithm used in establishing a shared secret key between two parties over a public channel. NTT is an analogue to the beautiful yet powerful Fast-fourier Transform (FFT).

This article is part one of two-part series titled **Fast-fourier Transform (FFT), Number-theoretic Transform (NTT) and Lattice-based Cryptography**. The goal of this post is to take a deep dive into FFT and NTT while part two will explore how NTT is used in lattice-based algorithms like [ML-KEM](#) and [ML-DSA](#).

As always, we are going to take a discovery approach, building up slowly from the familiar to the unfamiliar.

Polynomial Multiplication

Given two polynomials A and B of degree 2 :

-
-

We are asked to multiply them. Easy, right?

-

Below is an implementation of this in Python:

```
A = [3, -4, 1] # coefficients of A
B = [6, -5, 1] # coefficients of B
d = 2 # degree

C = [0 for _ in range((2 * d) + 1)]

for i in range(d + 1):
    for j in range(d + 1):
        C[i + j] = A[i] * B[j]
```

This algorithm takes time. Is there a faster way to perform polynomial multiplication?

From the implementation above, you can tell that the polynomial is represented as a list of its *coefficients*. There are other representations.

According to the **uniqueness theorem**, a polynomial of degree at most is uniquely determined by its values at **distinct** points. That is, a polynomial of degree can be uniquely constructed by the points: . Let's call this representation **point representation**.

A more general version of the uniqueness theorem states that a polynomial of degree at most is uniquely determined by its values at *at least* **distinct** points. That means we could uniquely represent our polynomial using more than points.

Now, let's represent our polynomials using the point representation, using the values . Therefore, our polynomial can be represented as follows:

Now, on to the interesting part. It turns out that . That is, if you multiply the evaluation by the evaluation , you get the evaluation . See the table below:

This means that the points uniquely represents the polynomial .

What does this all mean? We just achieved an time polynomial multiplication by using the point representation form of the polynomials.

Does that mean we just achieved for polynomial multiplication? Not really.

Given two polynomials $p(x)$ and $q(x)$ of the form: $p(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$, here's the full process:

1. **Convert from coefficient to point representation:** we pick a set of m values and evaluate the polynomials at those points thereby converting the polynomials to the point representation.

That is, we pick $x_1, x_2, \dots, x_m$. Then, we compute $p(x_i)$ and $q(x_i)$ for $i = 1, 2, \dots, m$.

There's a way of expressing this in a matrix form:

where n is at least M , V is the matrix of x_j and their powers, c is a vector of coefficients and y_j are the evaluations. V is called the **vandermonde** matrix.

This shows that the process of evaluating the polynomial $p(x)$ is a *matrix-vector multiplication* which is an $O(M^2)$ operation. y_j can also be written as follows:

Keep this in mind, as this formula for expressing y_j is important in understanding FFT later.

Using $p(x) = x^2 + 1$ as an example, choosing $M = 4$ and $x_j = \omega_4^j$, we have the following:

2. **Multiply pairwise** (i.e. $\text{poly}(n)$). This runs in $O(n^2)$ time.

3. **Convert back from the point representation to the coefficient representation:** This is the reverse of step one. We want to find $a(x)$, given y_k and ω_n^k . To compute this, we find the **inverse of** ω_n denoted by ω_n^{-1} and multiply by ω_n^{-1} . That is:

Using $n=4$ as an example:

The best version of this algorithm runs in $O(n \log n)$ time.

In total, this process still takes $O(n \log n)$ time.

But, what if there was actually a faster way? A way that allows us to do polynomial multiplication in less than $O(n^2)$.

Fast-fourier Transform (FFT) gives us that. FFT allows us to do step one and step three in $O(n \log n)$ time; making the whole process $O(n \log n)$ time.

To understand FFT, we need to understand the concept of **complex numbers** and **roots of unity**

Complex Numbers and Roots of Unity

The magic of FFT is based on the beautiful concept of roots of unity but that in turn is only made possible by the unique properties of complex numbers. So, here's a deep dive on complex numbers and roots of unity.

Given the polynomial $x^2 + 1$, find the roots (i.e, the values of x for which $x^2 + 1 = 0$) of the polynomial. The simple answer is the roots does not exist.

Using the quadratic formula $\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$, let's solve anyways:

- $x = \frac{-0 \pm \sqrt{0^2 - 4(1)(1)}}{2(1)}$ and $x = \frac{-0 \pm \sqrt{0^2 - 4(1)(1)}}{2(1)}$
- $x = \frac{-0 \pm \sqrt{0^2 - 4(1)(1)}}{2(1)}$ and $x = \frac{-0 \pm \sqrt{0^2 - 4(1)(1)}}{2(1)}$

Therefore, the roots of $x^2 + 1$ are $x = \frac{-0 \pm \sqrt{0^2 - 4(1)(1)}}{2(1)}$ and $x = \frac{-0 \pm \sqrt{0^2 - 4(1)(1)}}{2(1)}$.

But, we previously said the polynomial didn't have a root. How come we got two roots? The answer lies in the fact that from the beginning we have been dealing with real numbers and the roots $x = \frac{-0 \pm \sqrt{0^2 - 4(1)(1)}}{2(1)}$ and $x = \frac{-0 \pm \sqrt{0^2 - 4(1)(1)}}{2(1)}$ are not real numbers. Therefore, the roots does not exist in real space.

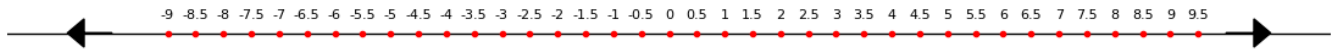
If the roots are not real numbers, then what are they? They are complex numbers!

Let's further simplify the roots. $\sqrt{-1}$ can be written $\sqrt{-1} = \sqrt{1 \cdot -1}$. But, we know that $\sqrt{-1}$ is impossible to find. It has a special name, the *imaginary unit* denoted as i .

We can rewrite our roots as follows: $x = \frac{-0 \pm \sqrt{0^2 - 4(1)(1)}}{2(1)}$ and $x = \frac{-0 \pm \sqrt{0^2 - 4(1)(1)}}{2(1)}$. Therefore, a complex number is a number of the form: $a + bi$ where a and b are real numbers and i is the imaginary unit. a is called the *real part* and bi is called the *imaginary part*. This form of representing a complex number is called the *rectangular form*. You'll see why soon.

The set of real numbers is a proper subset of the complex numbers (i.e, $\mathbb{R} \subset \mathbb{C}$). Every real number can be written as a complex number where $b = 0$ (e.g $5 = 5 + 0i$).

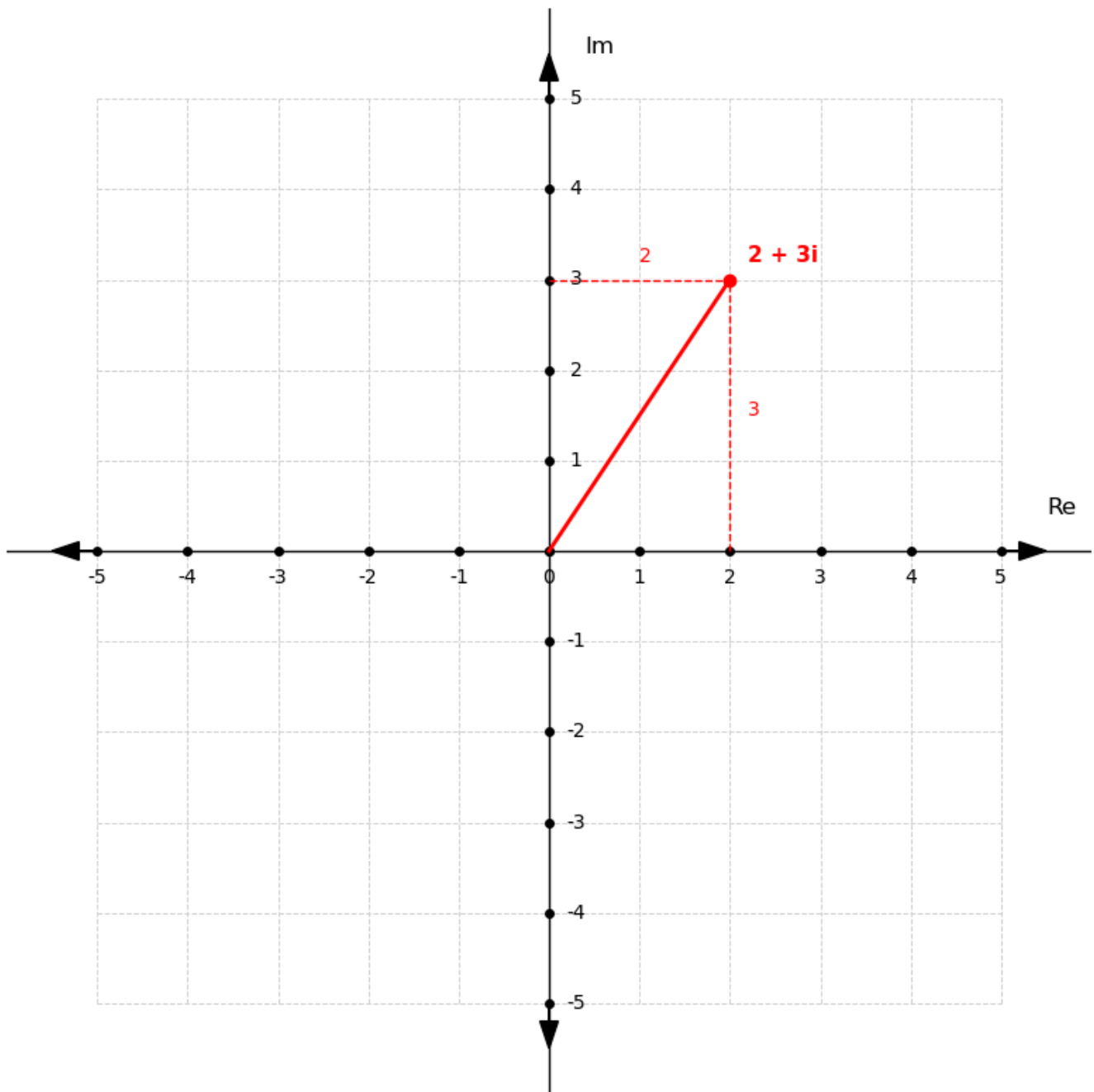
One way to describe a real number is as follows: a real number is any number that can be located on the infinite number line. That means, real numbers are represented on a number line as shown below.



How do we represent a complex number? The number line would have been a good option but it would only be able to take the real part. We need diagram that allows us to show both the real part and the imaginary part. That's where the **Argand diagram** comes in!

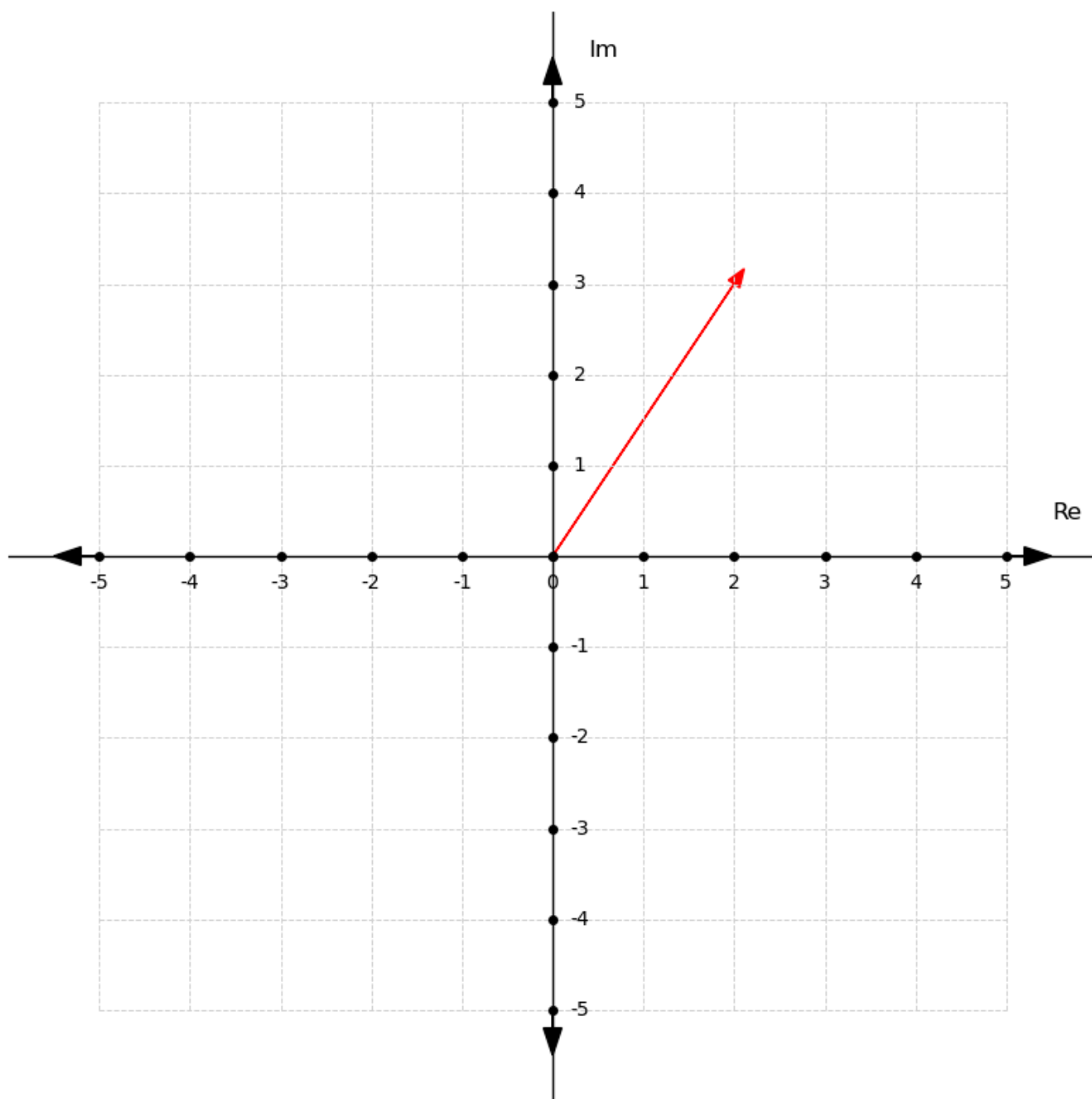
The Argand diagram is two dimensional plane with the real part on the horizontal line(x-axis) and the imaginary part on the vertical line (y-axis).

For example, the complex number is represented as follows:

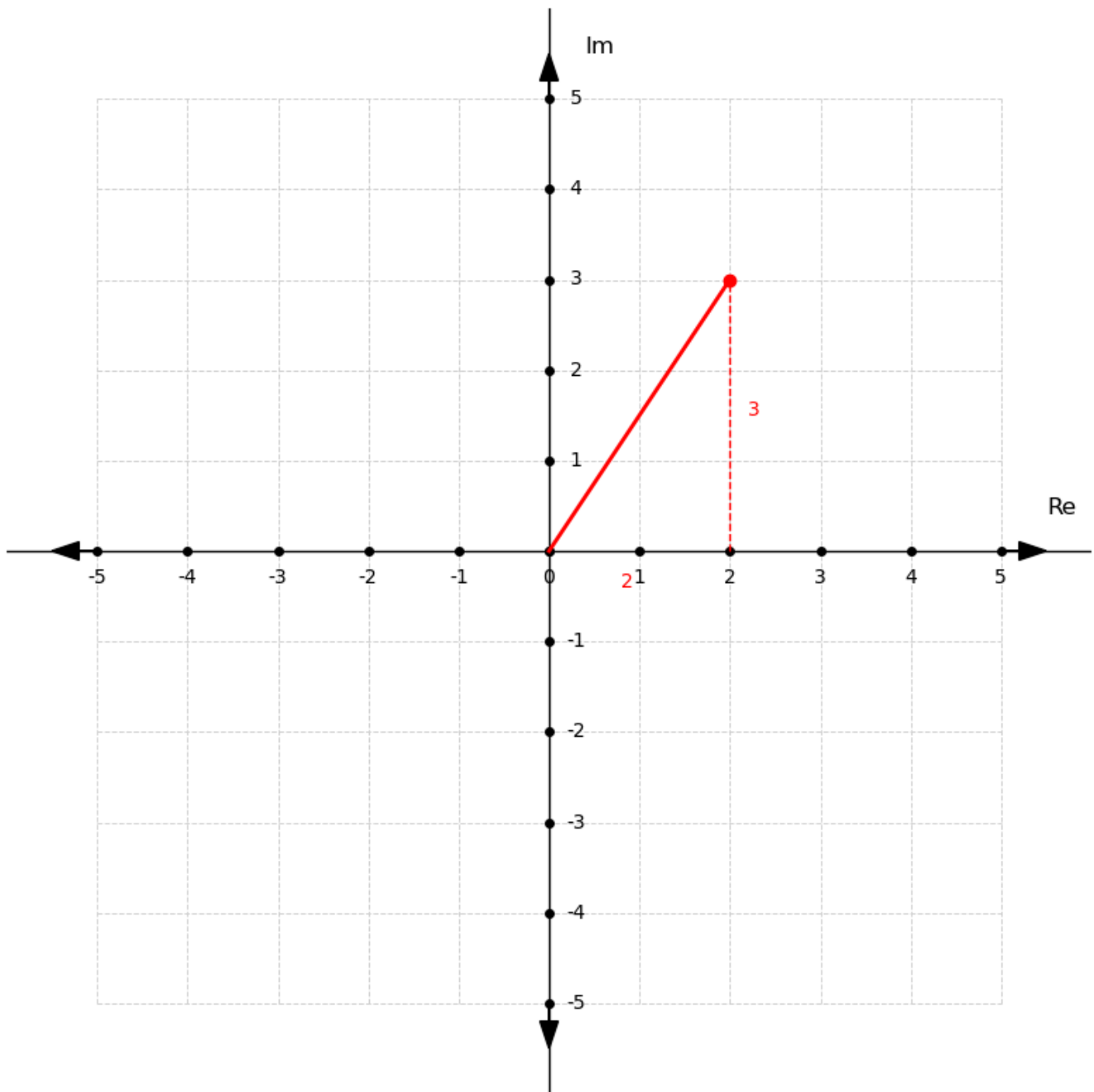


Notice how the dotted lines from 2 and 3 forms a rectangle. That's why the form $a + bi$ is called the *rectangular form*.

Imagine we were given the diagram with just the line from the origin as shown below. Would we be able to identify the complex number?



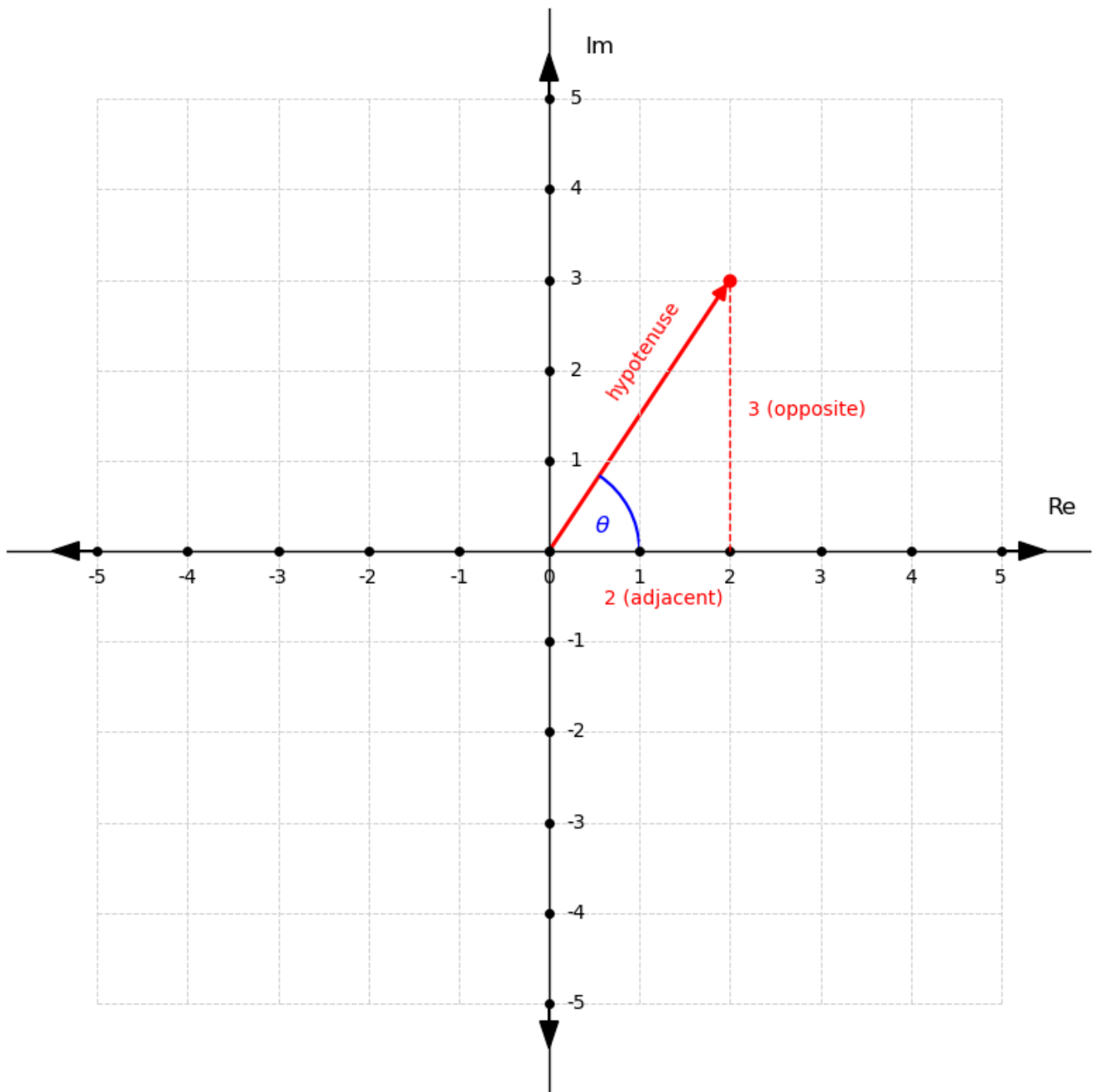
Notice how the angle forms a *right angle triangle*. The diagram below should help.



What do we know about right angle triangles?

According to the **Pythagoras Theorem**, in a right angled triangle, the square of the length of the hypotenuse (the side opposite the right angle) is equal to the sum of the squares of the lengths of the other two sides (*opposite* and *adjacent*). Mathematically, if a right-angled triangle has sides a and b , and hypotenuse c , then: $a^2 + b^2 = c^2$.

Having known this; let's adjust our diagram.



From the diagram, we can see that we have two unknowns, the hypotenuse which will be denoted as r and the angle θ .

From pythagoras theorem, we know $r^2 = 2^2 + 3^2$. Therefore:

—

Now we know r , how do we find θ ? Do you remember the acronym SOHCAHTOA from high school? This implies:

$$\begin{aligned} &= \frac{1}{2} \left(\frac{1}{2} + \frac{1}{2} \right) + \frac{1}{2} \left(\frac{1}{2} + \frac{1}{2} \right) \\ &= \frac{1}{2} + \frac{1}{2} = 1 \end{aligned}$$

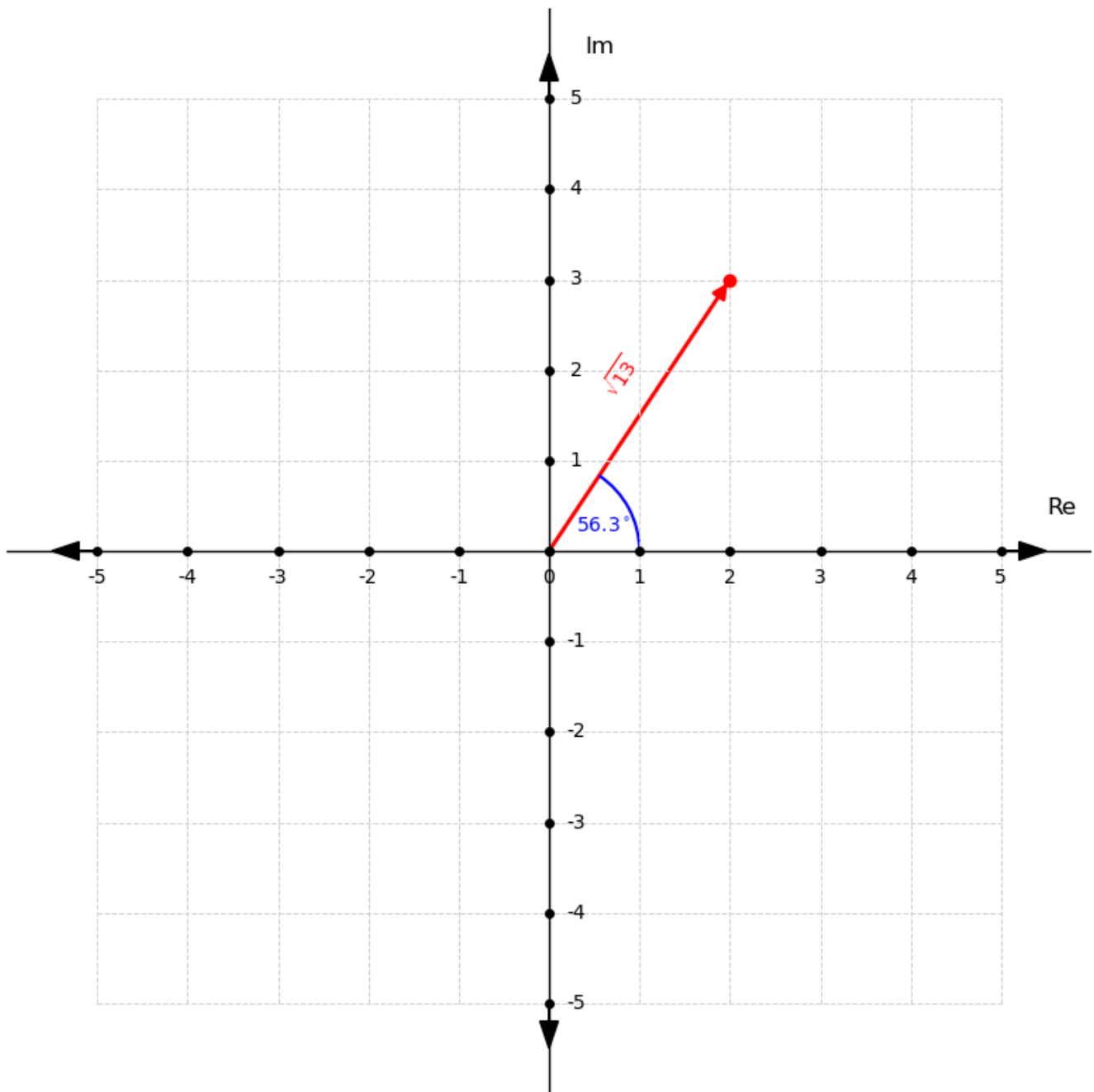
From this, we can deduce that: $z = r(\cos \theta + j \sin \theta)$. This form of representing a complex number is called the *polar form*.

Now we find r using $r = \sqrt{a^2 + b^2}$, $\theta = \tan^{-1} \frac{b}{a}$ radians.

We have that $r = \sqrt{a^2 + b^2}$ and $\theta = \tan^{-1} \frac{b}{a}$. Given these two values, we represent the complex number as $r(\cos \theta + j \sin \theta)$. θ radians is $\theta \times \frac{180}{\pi}$ in degrees and it's better for visualization as shown below.

The polar representation might not look like the simplest way to express a complex number but it's going to help us understand roots of unity.

Formally, r is distance from the point of the complex number to the origin and it is called the *modulus* or the *absolute value*. θ is called the *argument* of the complex number and it is the angle that the modulus r makes from the positive real axis.



Let's look at another representation of complex numbers.

Have you ever wondered how calculators computed trigonometric functions like $\sin$, $\cos$ and the exponential function e^x ? The answer lies in approximation using power series.

$\sin$ and $\cos$ are approximated using the power series $x - \frac{x^3}{6} + \frac{x^5}{120} - \frac{x^7}{5040} + \dots$ and $1 - \frac{x^2}{2} + \frac{x^4}{24} - \frac{x^6}{720} + \dots$ respectively. I urge you to verify this using a graph(*i.e* compare their graphs against each other while extending the series).

Let's substitute i for x :

$$\frac{1}{1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \dots}$$

Now, let's substitute this approximation into the polar form:

$$\frac{1}{1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \dots} = \frac{1}{1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \dots}$$

Let's look at that of the exponential function . The power series for as follows:

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \frac{x^5}{5!} + \frac{x^6}{6!} + \frac{x^7}{7!} + \frac{x^8}{8!} + \dots$$

Let's substitute for :

$$e^{\frac{1}{1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \dots}}$$

We will compute the powers of . From indices we know that . Let's focus on :

- $\frac{1}{1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \dots}$
- $\frac{1}{1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \dots}$
- $\frac{1}{1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \dots}$
- $\frac{1}{1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \dots}$
- $\frac{1}{1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \dots}$
- $\frac{1}{1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \dots}$
- $\frac{1}{1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \dots}$
- $\frac{1}{1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \dots}$
- $\frac{1}{1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \dots}$

Do you see the pattern? The values are cyclic in nature; to , it repeats and continues like that. We will come back to this but for now let's go back to computing . It now becomes:

$$\frac{1}{1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \dots}$$

From this we can see that . Therefore, we have our third representation, the *exponential form*: .

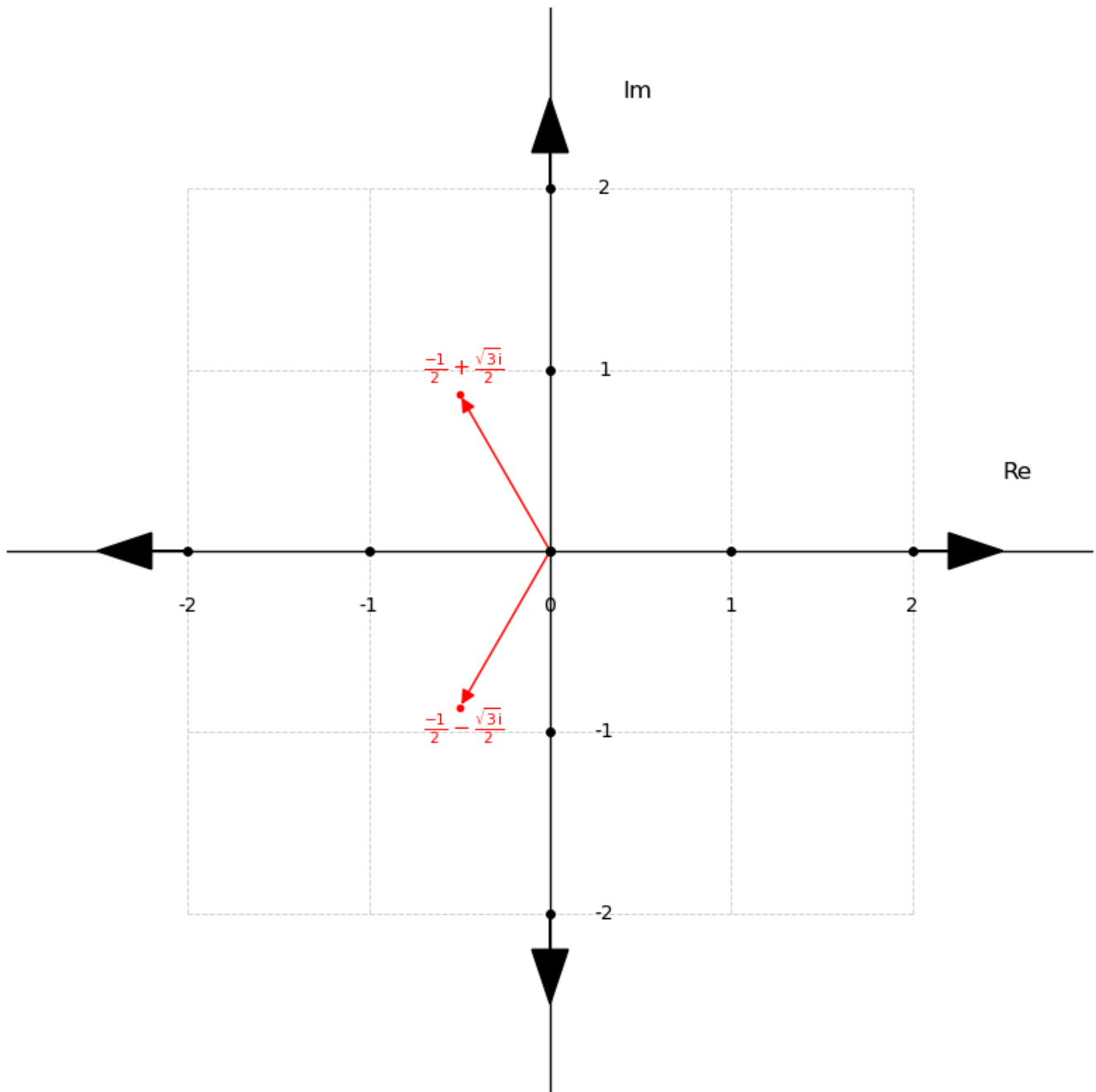
In summary, we have:

- rectangular form:
- polar form:
- exponential form:

Soon, you will see why exponential and polar form are best suited for understanding and working with roots of unity.

Now, back to where this it all started: finding the roots of the polynomial $x^3 - 1 = 0$. We saw that the roots are 1 , ω and ω^2 .

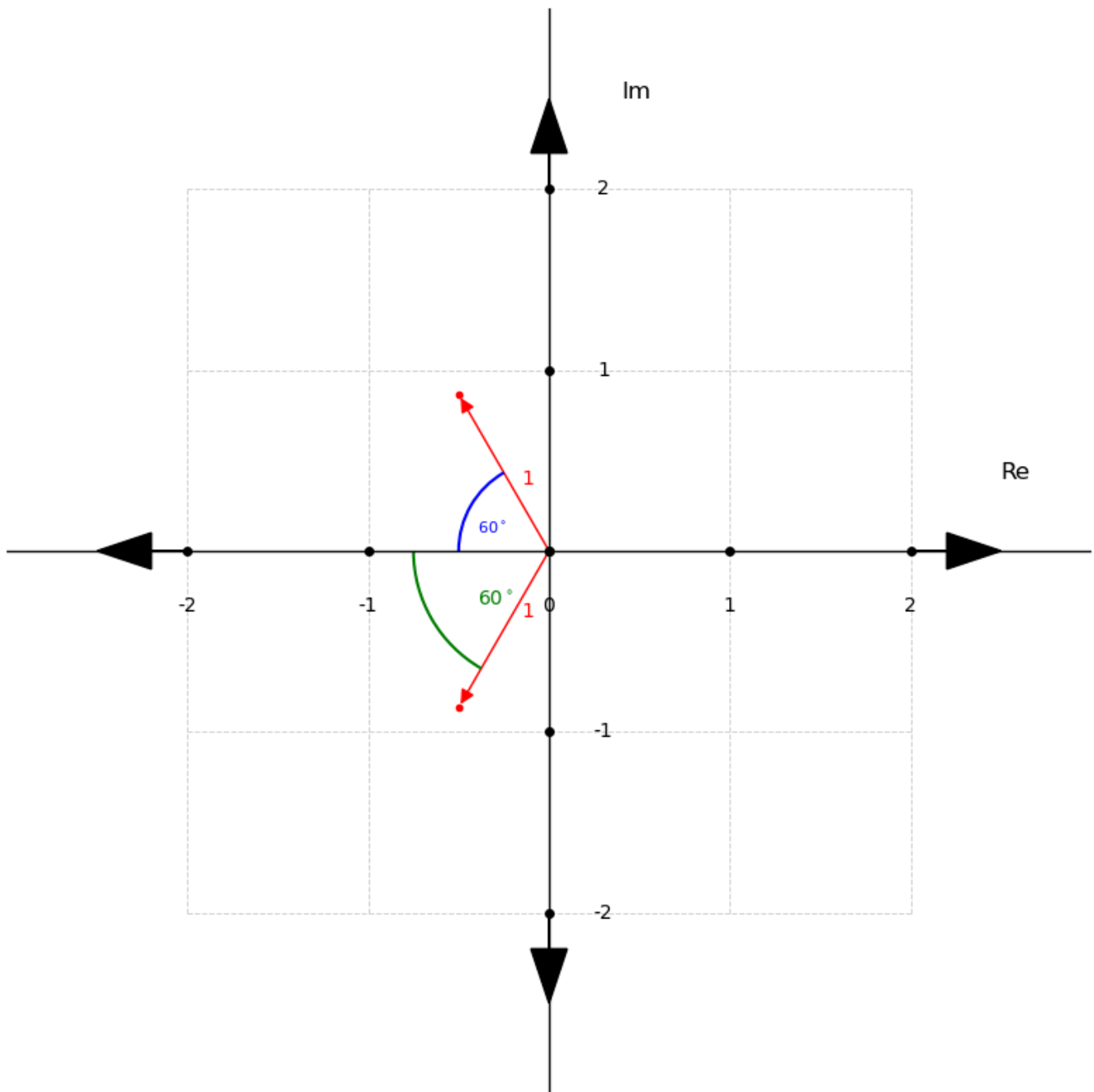
Let's represent them using the argand diagram.



Let's find the modulus and the argument of these roots:

- For — —:
- finding : , — , — ,
— — — — ,
—
— or —
- For — —:
- finding : , — , — ,
— — — — ,
—
— or —

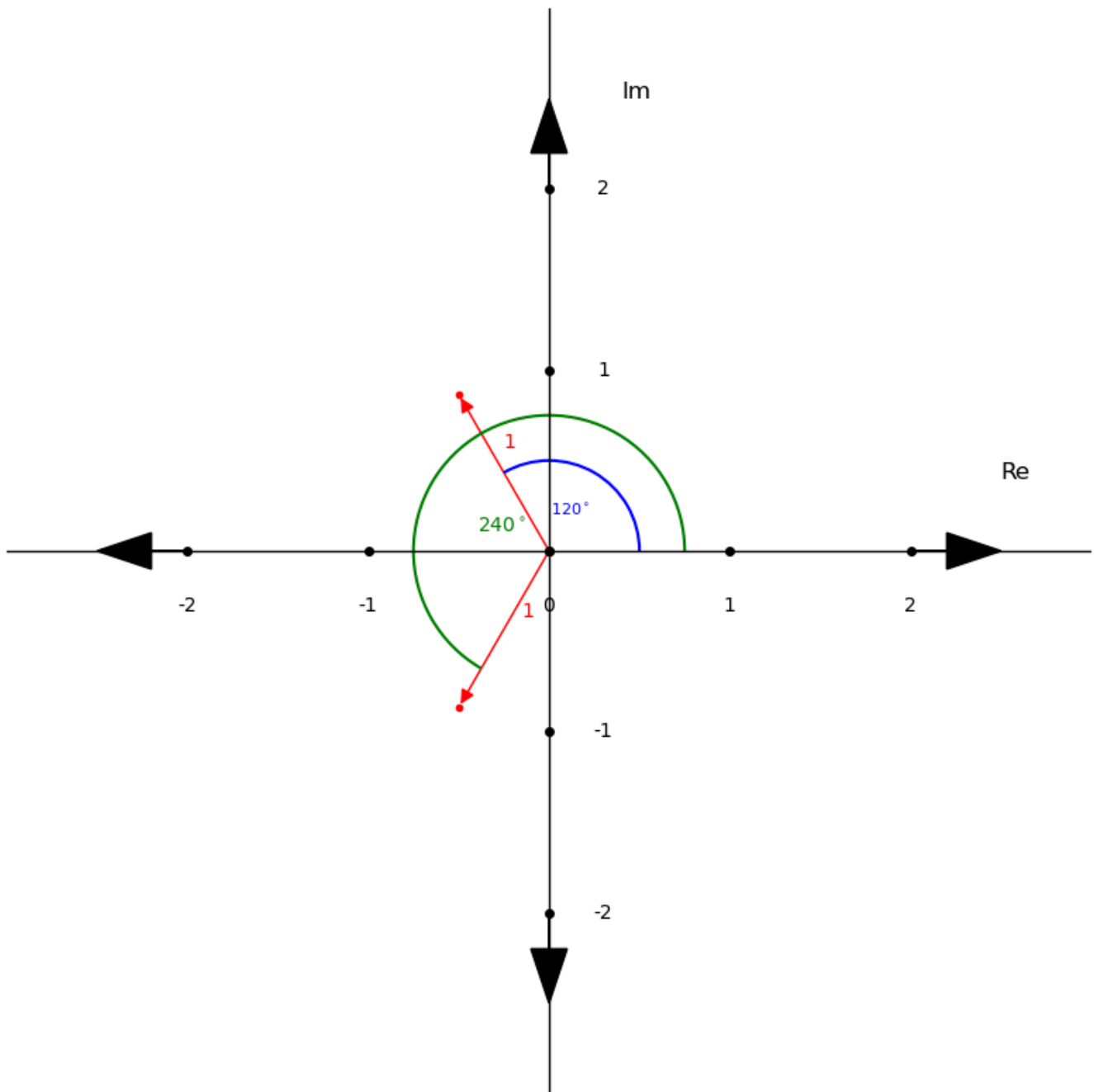
Let's show our α and β values to our diagram.



This is our argand diagram represented in respect to Re and Im . But, it's not exactly correct.

Recall that the argument is "the angle that the modulus makes from the positive real axis". The key phrase is *from the positive real axis*.

Let's show this:



As you can see, for $\omega = e^{j120^\circ}$ becomes ω^2 or e^{j240° and for $\omega^2 = e^{j240^\circ}$ becomes ω or e^{j120° .

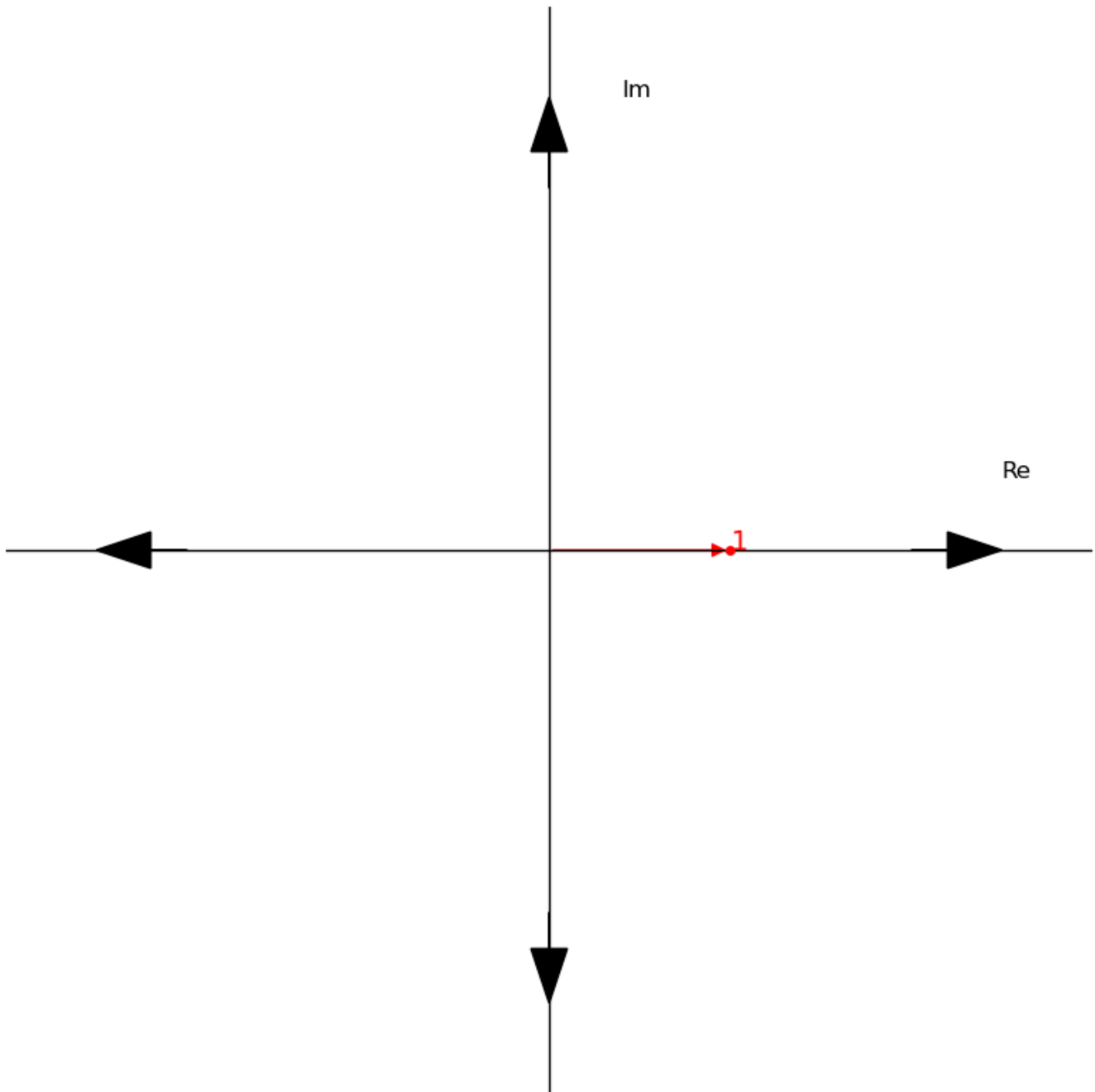
In exponential form, this will be written as e^{j120° and e^{j240° respectively as ω and ω^2 .

This also helps us see the argument not just as angle but a *rotation* from the positive real axis and this is the mental model we need to understand roots of unity.

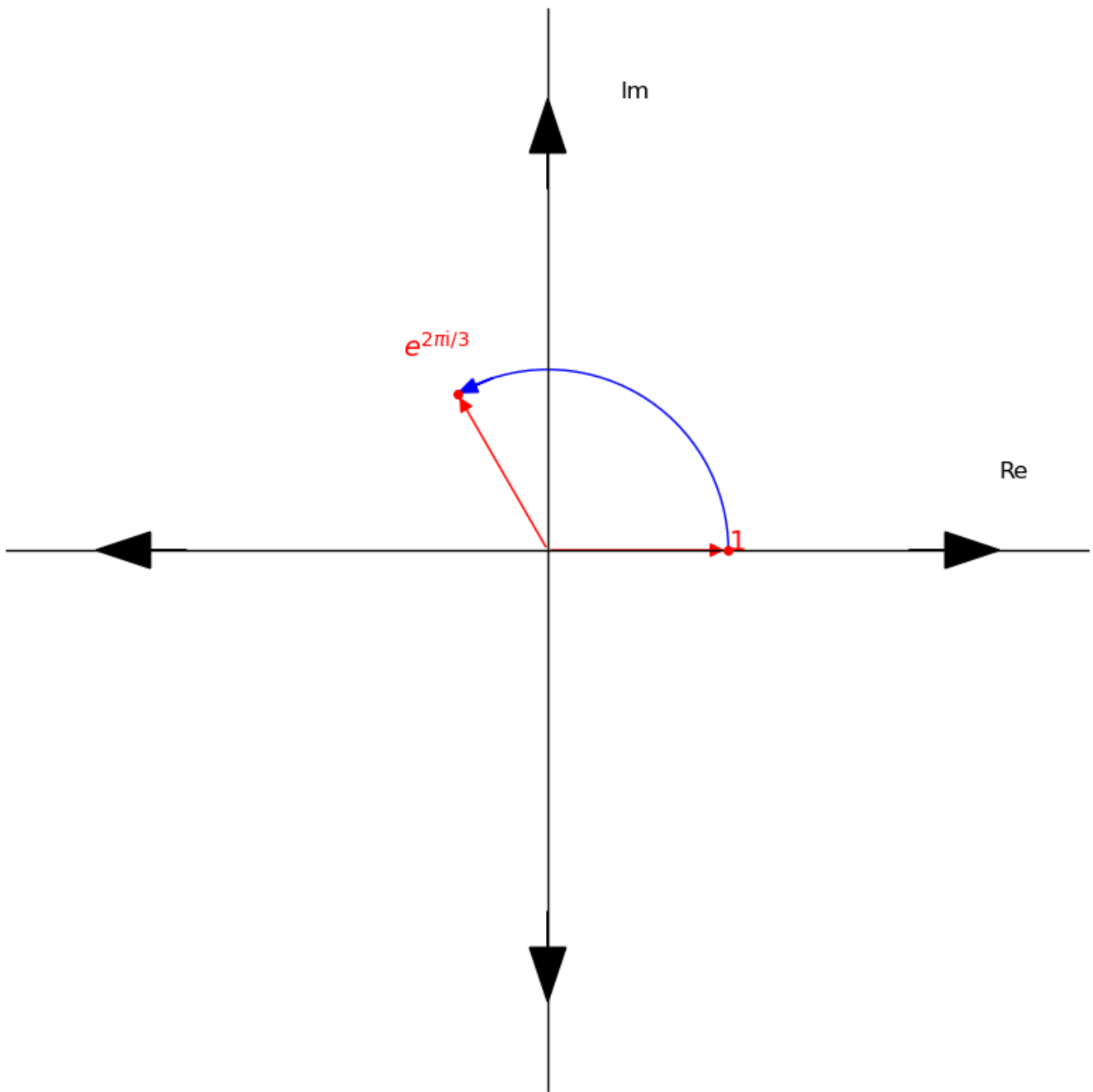
Let's do a quick experiment: we are going to take the roots individually and compute their powers up to the third power. Then, we are going to see what happens in the argand diagram.

Starting with :

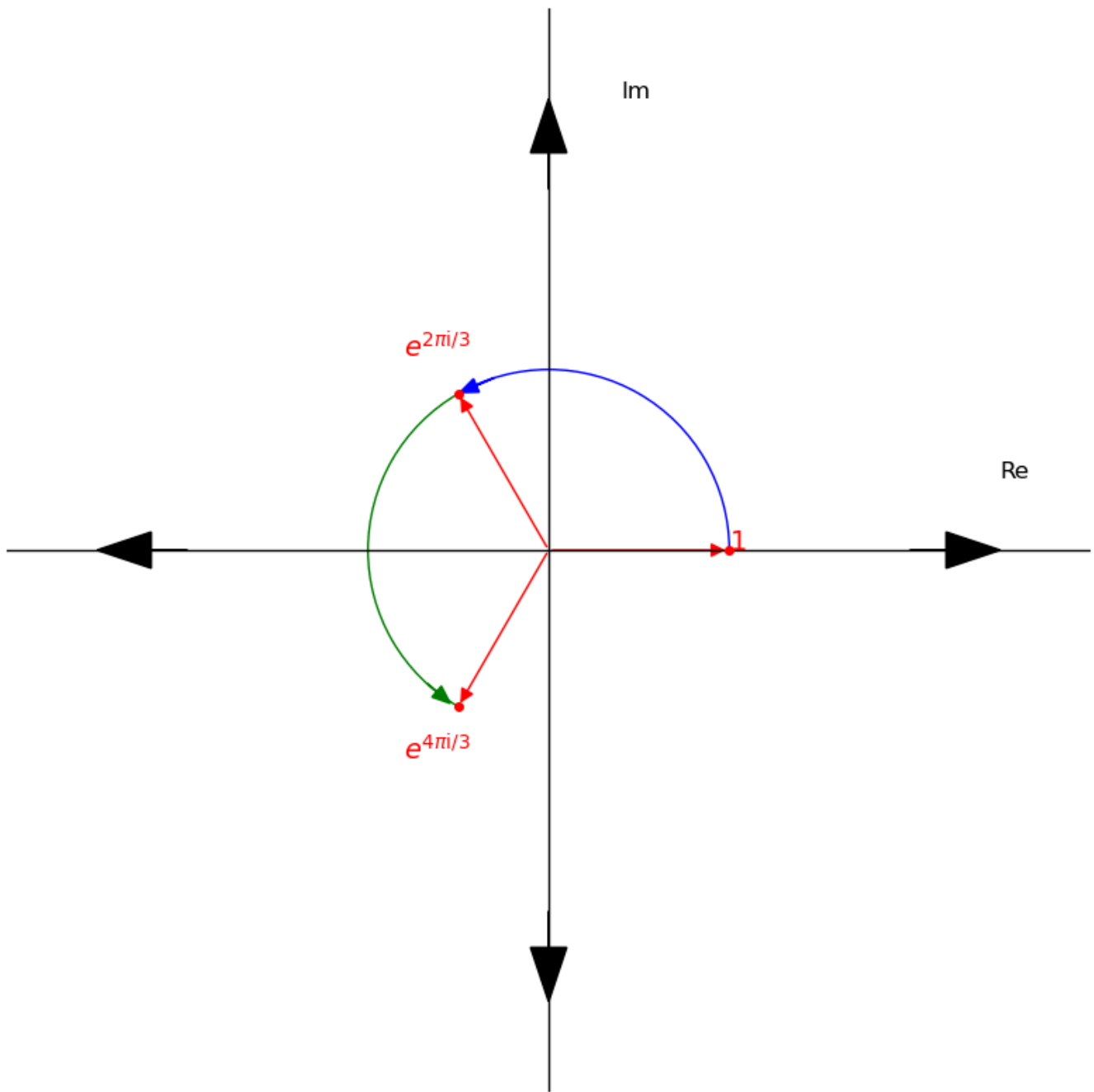
- We compute the zero power:



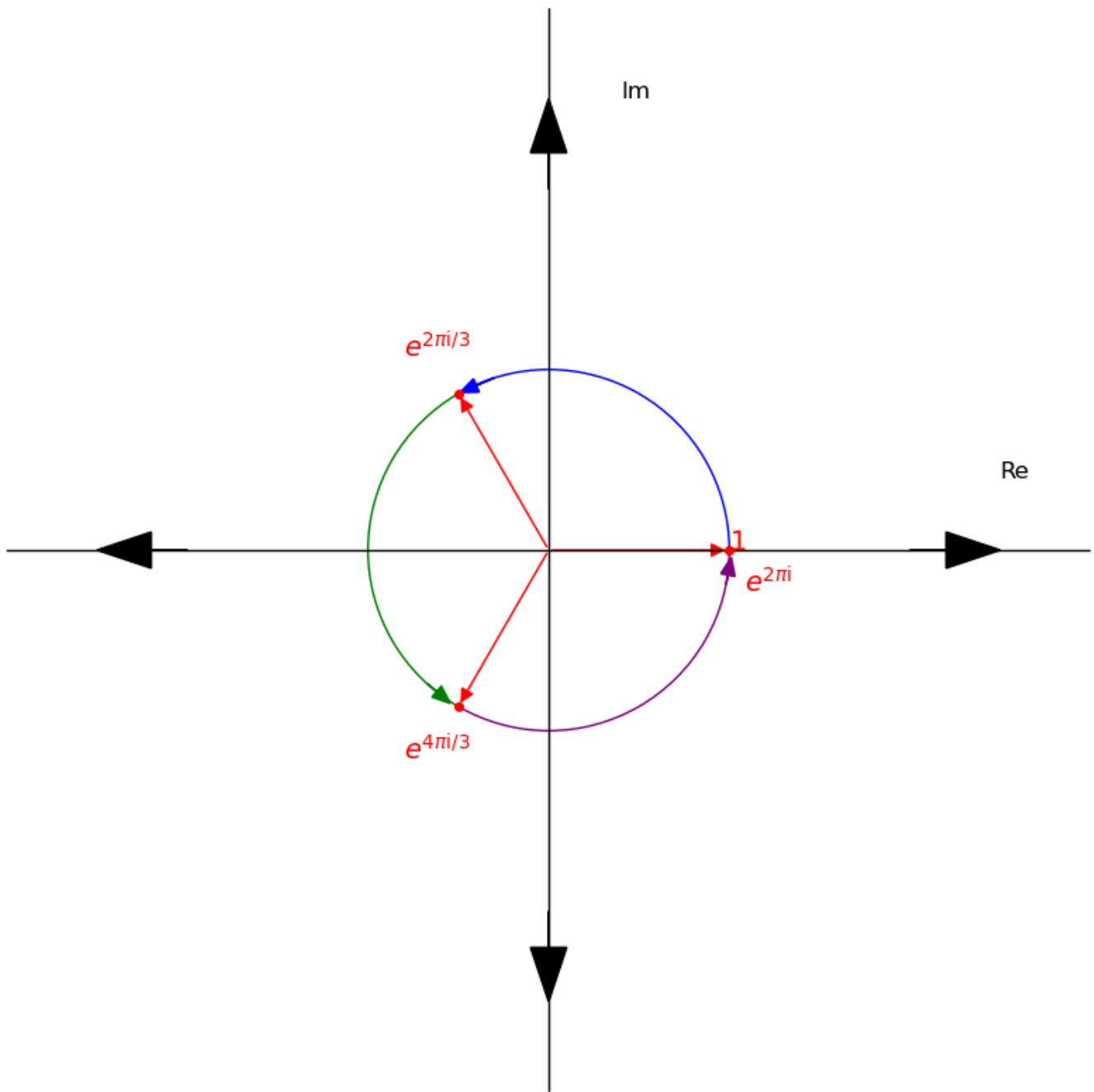
- The first power:



- The second power

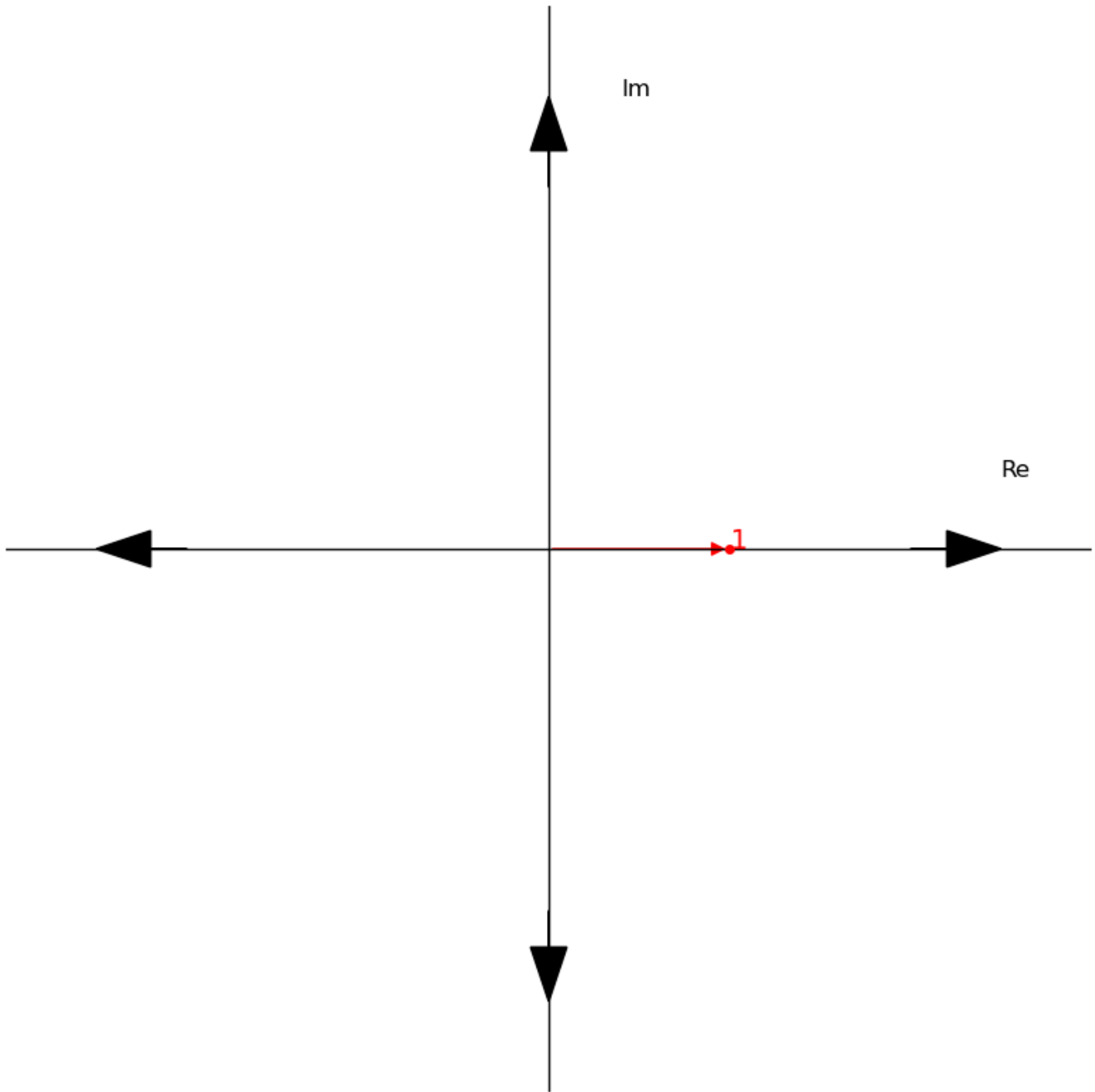


- The third power

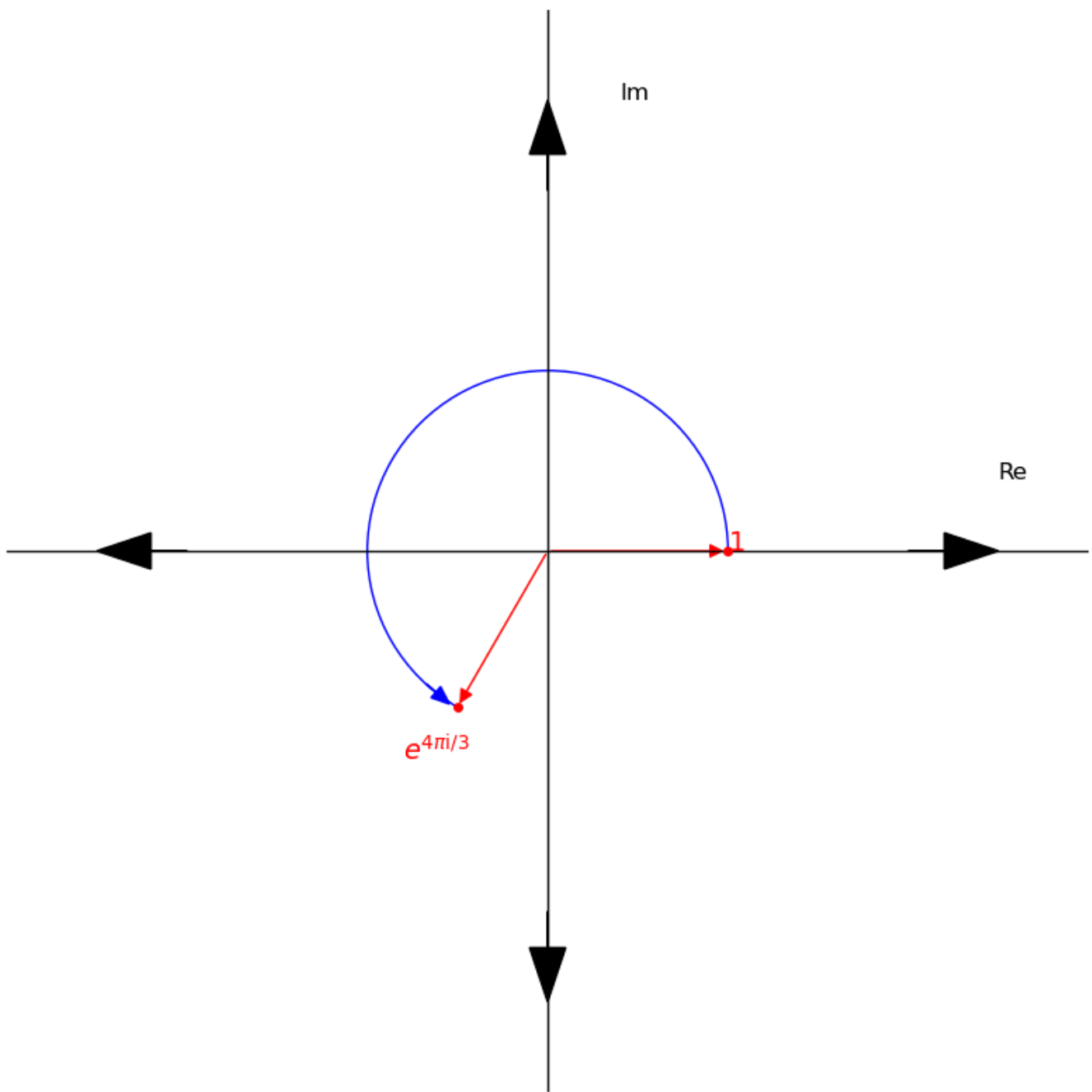


Then, for :

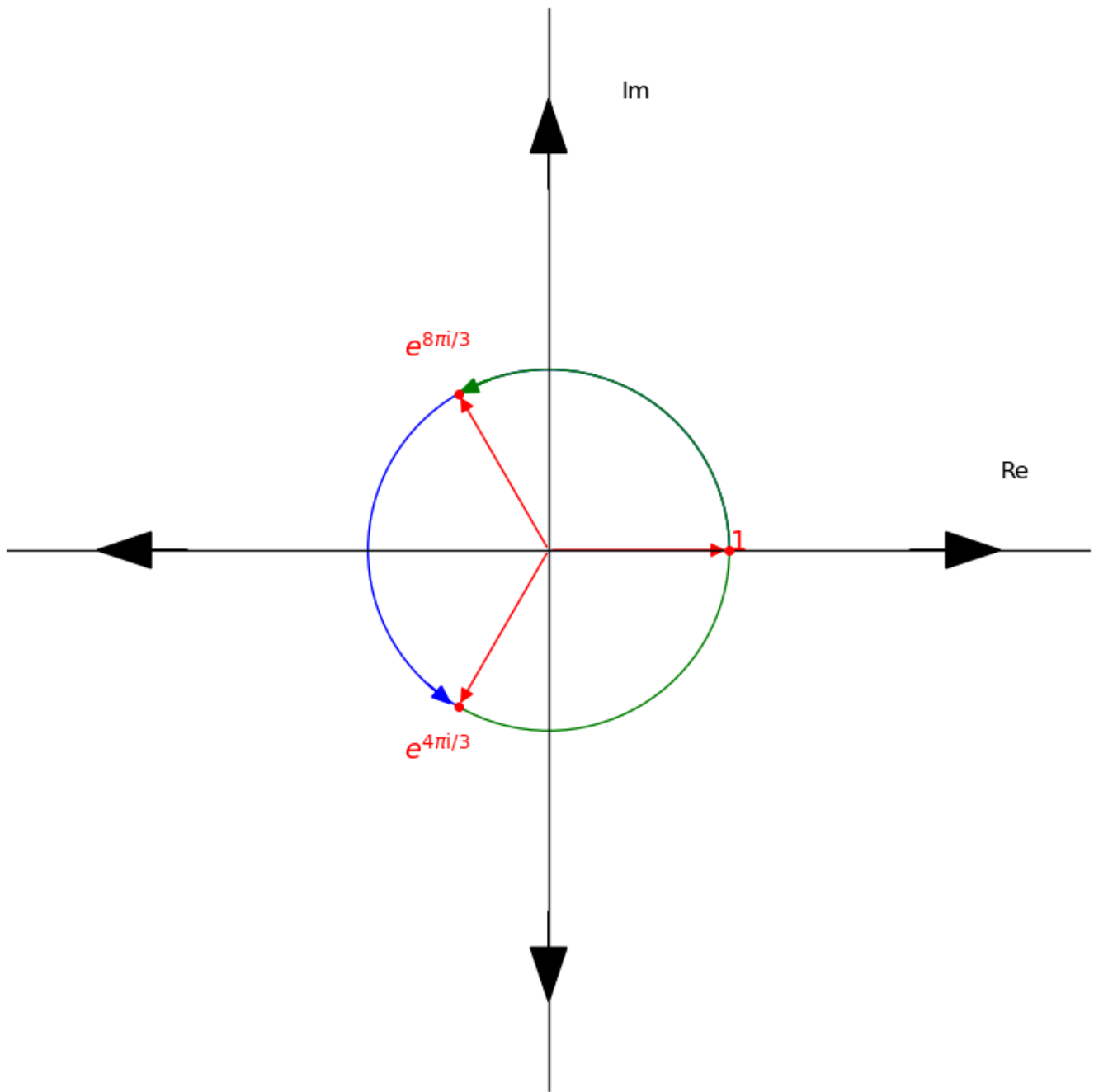
- We compute the zero power:



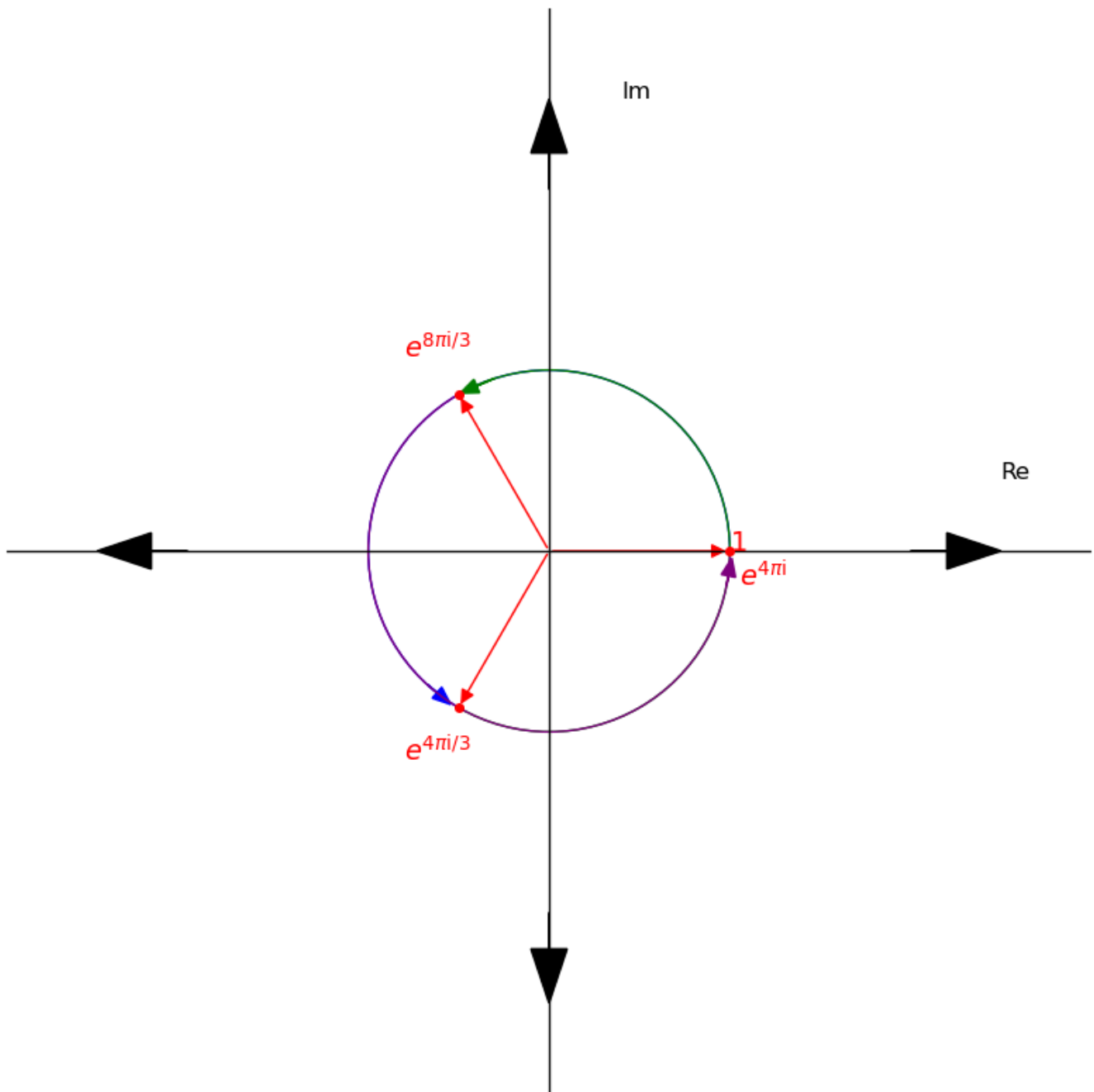
- The first power:



- The second power:



- The third power:



What do you notice?

The powers cycle back to 1 at the third power. If we continued this cycle, you would notice that it doesn't just cycle back to 1 at the third power but every power that's a multiple of 3 . I urge you to try this out.

Let's try this again for another polynomial.

Given the polynomial $x^3 - 1$, let's find the roots.

We get that:

•

Setting them to zero, we have that:

•

,

•

,

•

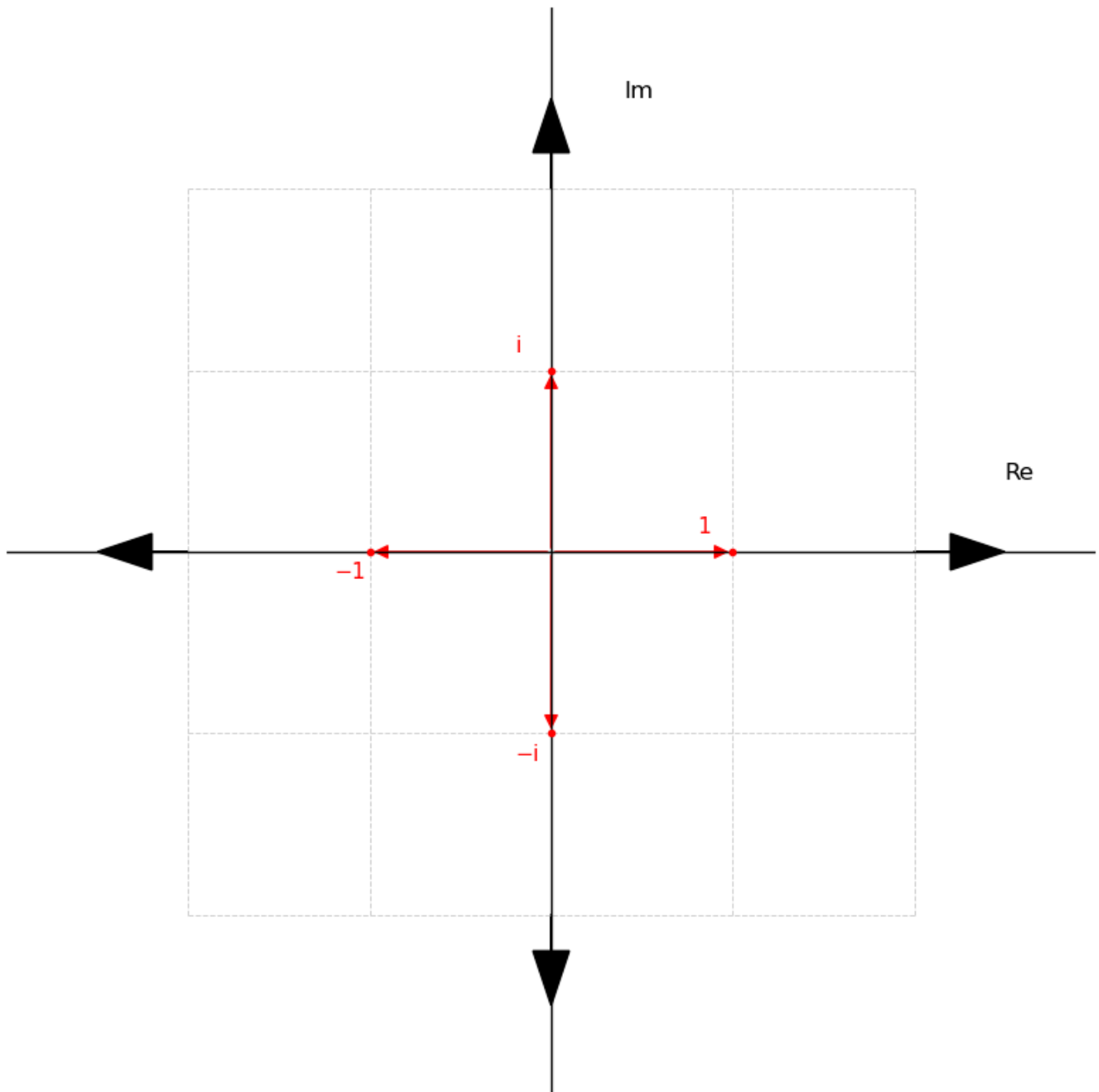
,

,

—

Therefore, the roots of are , , and

Let's show them on the argand diagram.



Let's compute the modulus and argument for each of them.

- For z_1 :
 - finding $|z_1|$: $\sqrt{1^2 + 1^2} = \sqrt{2}$, $\arg(z_1) = \frac{\pi}{4}$,
 - finding $\arg(z_1)$: $\frac{\pi}{4}$ or $\frac{5\pi}{4}$.
- For z_2 :
 - finding $|z_2|$: $\sqrt{1^2 + 1^2} = \sqrt{2}$, $\arg(z_2) = \frac{3\pi}{4}$,
 - finding $\arg(z_2)$: $\frac{3\pi}{4}$ or $\frac{7\pi}{4}$. from the positive x-axis is $\frac{3\pi}{4}$ so or
- For z_3 :
 - finding $|z_3|$: $\sqrt{1^2 + 1^2} = \sqrt{2}$, $\arg(z_3) = \frac{5\pi}{4}$,
 - finding $\arg(z_3)$: $\frac{5\pi}{4}$ or $\frac{\pi}{4}$. This looks like we are dividing 1 by 0 but trust me that's not the case. I will leave you to verify this yourself.
- For z_4 :
 - finding $|z_4|$: $\sqrt{1^2 + 1^2} = \sqrt{2}$, $\arg(z_4) = \frac{7\pi}{4}$,
 - finding $\arg(z_4)$: $\frac{7\pi}{4}$ or $\frac{3\pi}{4}$. from the positive x-axis is $\frac{7\pi}{4}$ so or

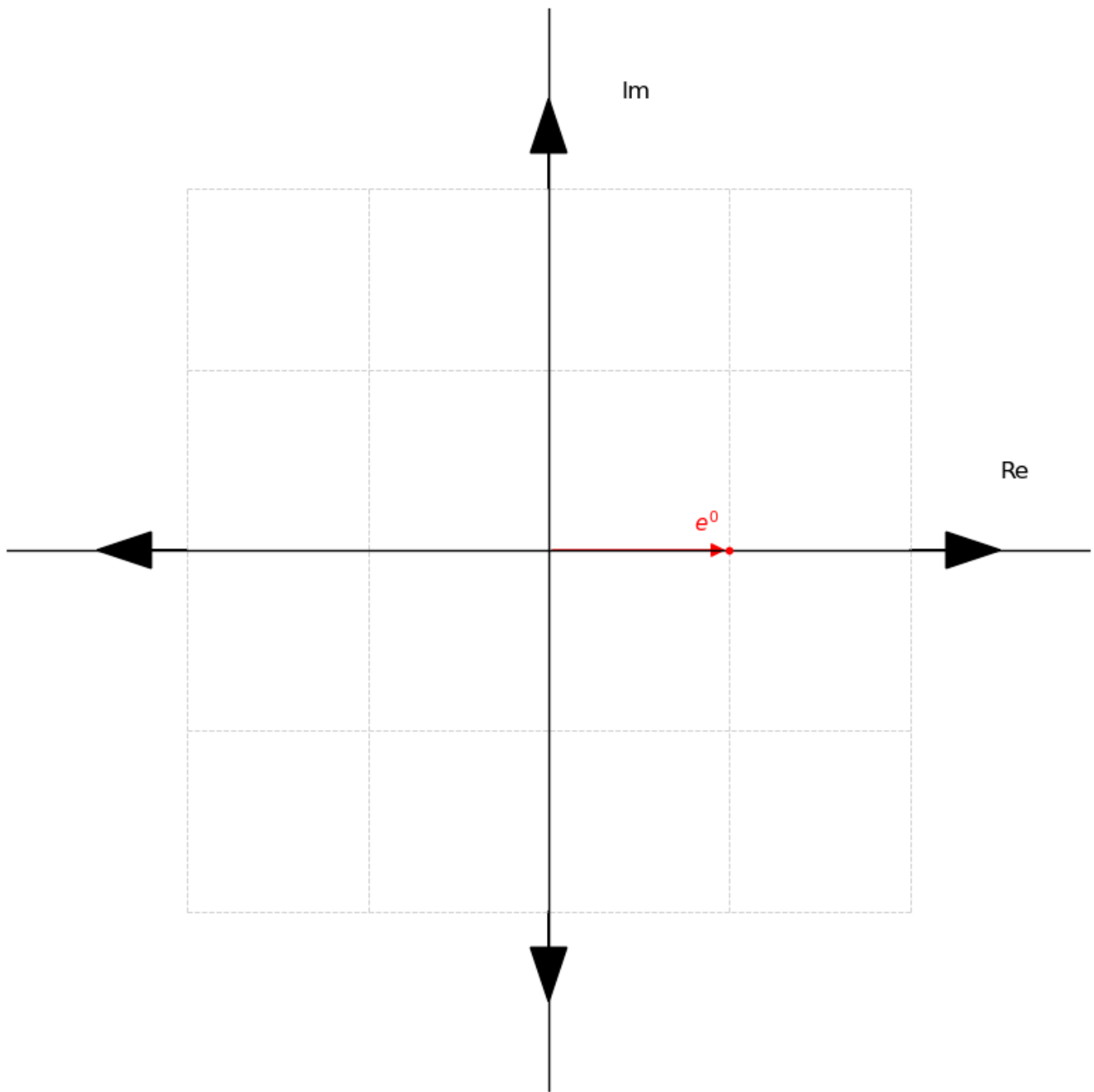
Converting them to exponential form, being that $e^{i\theta} = \cos(\theta) + i\sin(\theta)$ for all roots, we have that:

- $z_1 = \sqrt{2} e^{i\frac{\pi}{4}}$
- $z_2 = \sqrt{2} e^{i\frac{3\pi}{4}}$
- $z_3 = \sqrt{2} e^{i\frac{5\pi}{4}}$
- $z_4 = \sqrt{2} e^{i\frac{7\pi}{4}}$

Now, let's compute the powers of each root up to the fourth power.

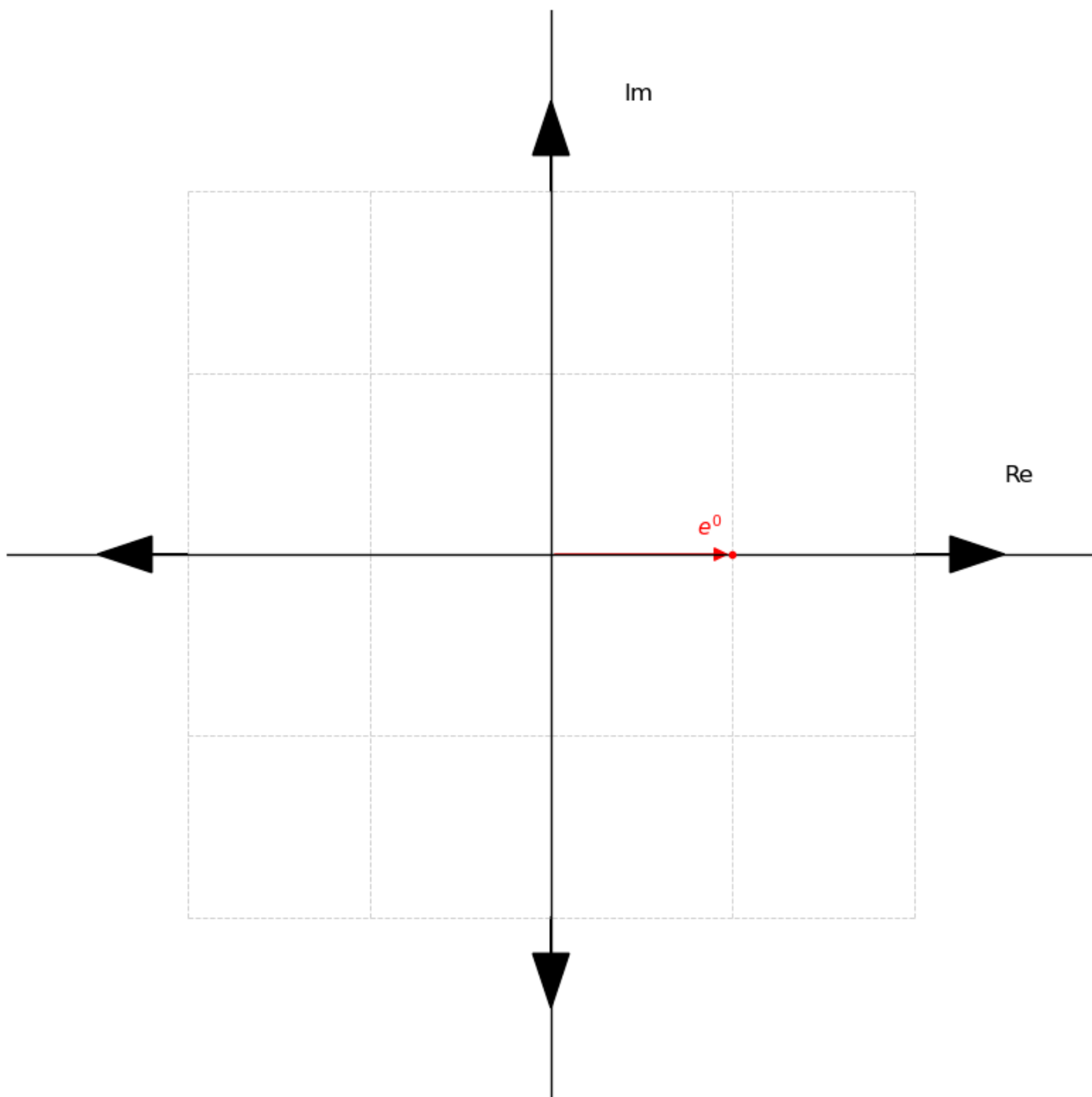
Starting with z_1 :

- We compute the zero power $z_1^0 = 1$. Notice that all the powers are equal as $z_1^4 = 1$ so we have just one argand diagram showing the point without rotation.

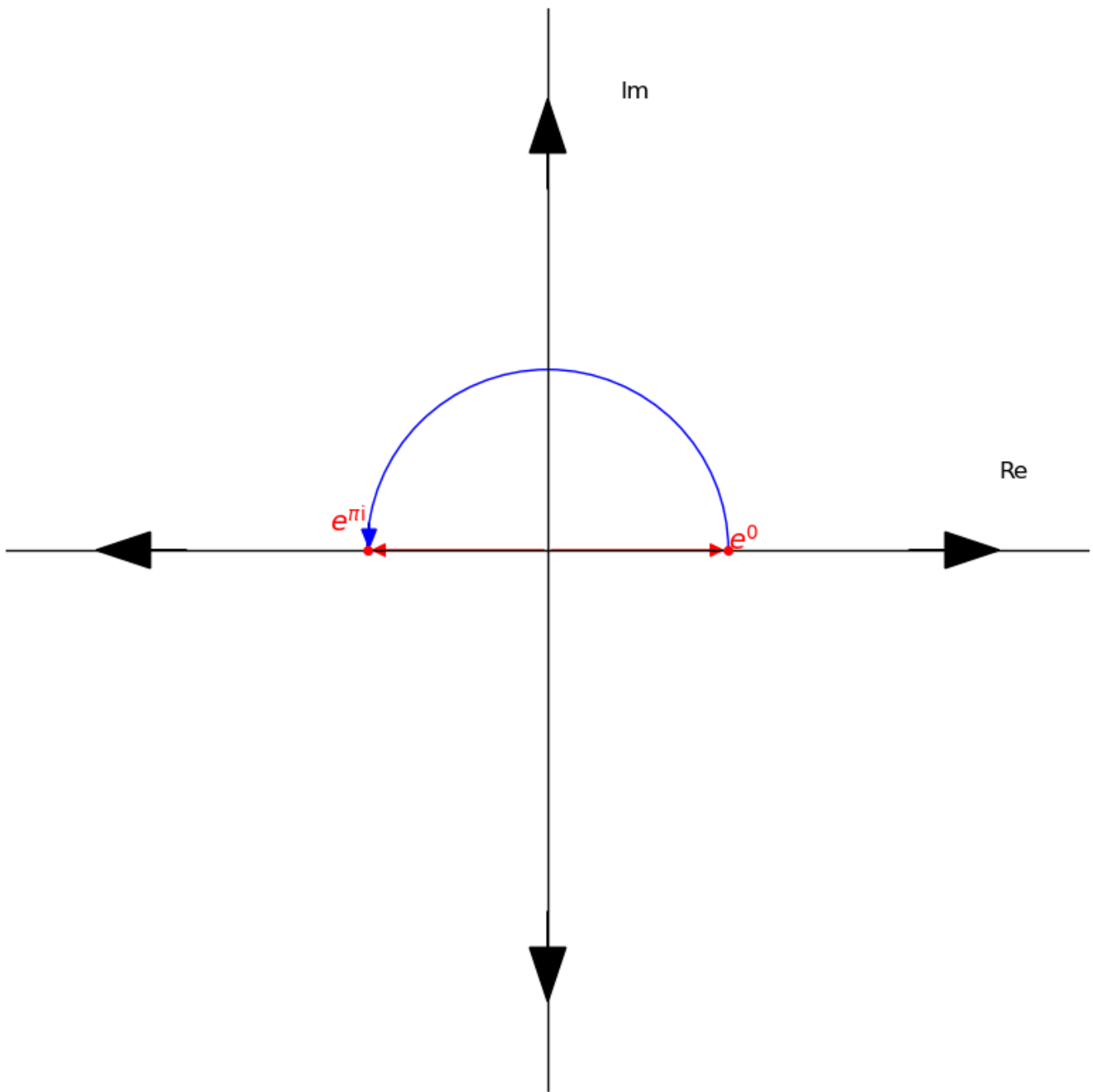


For :

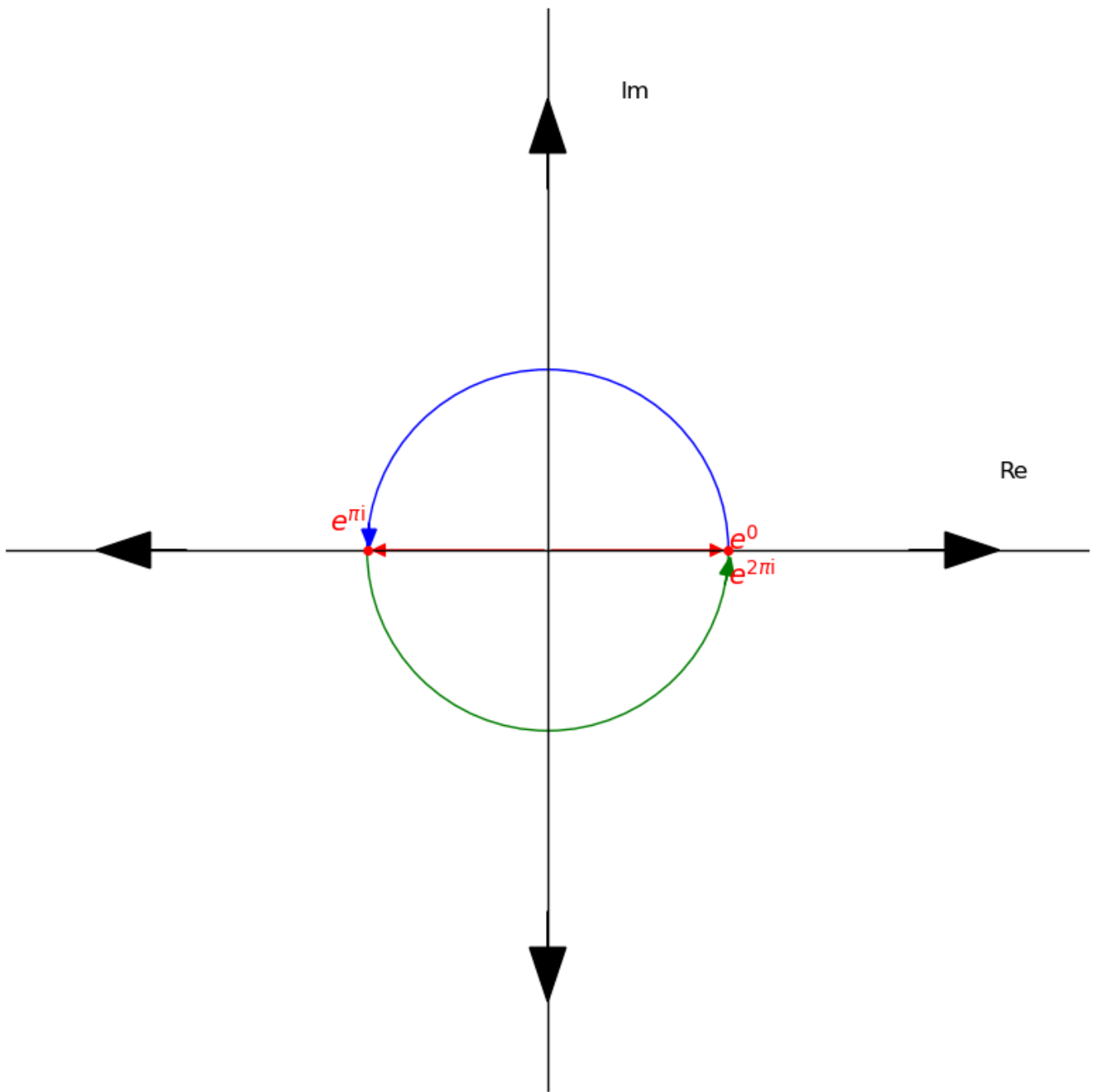
- We compute the zero power:



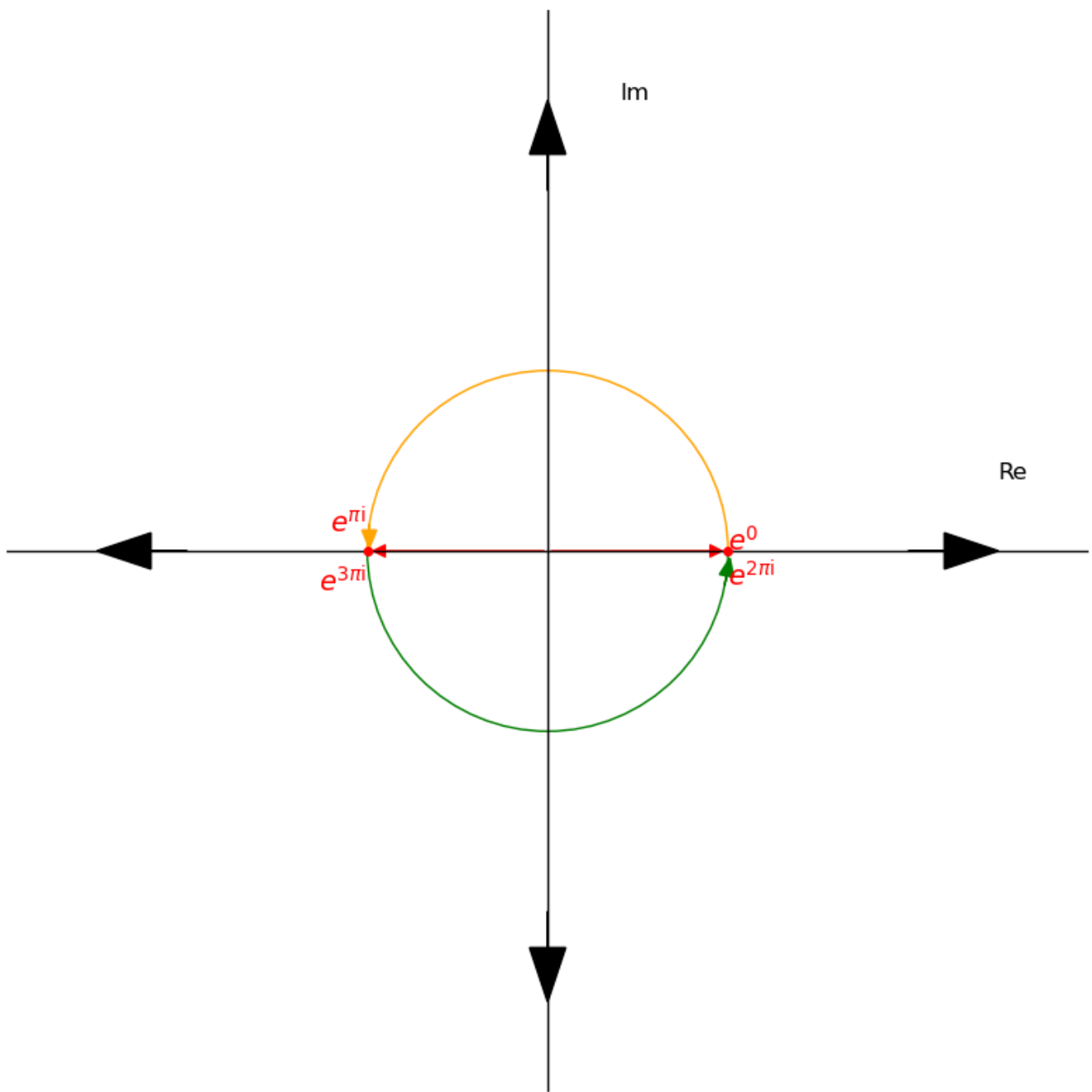
- The first power:



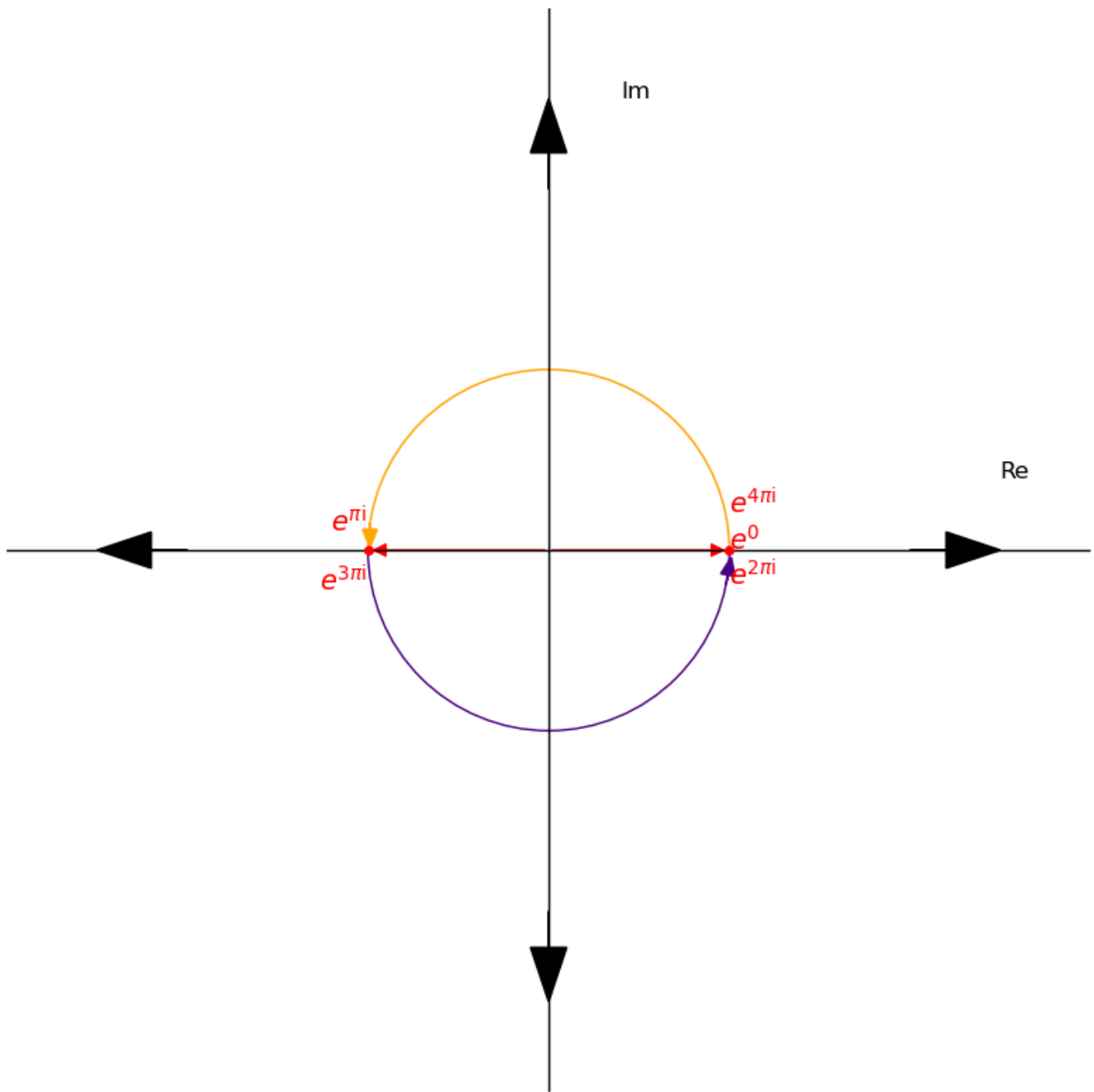
- The second power:



- The third power:

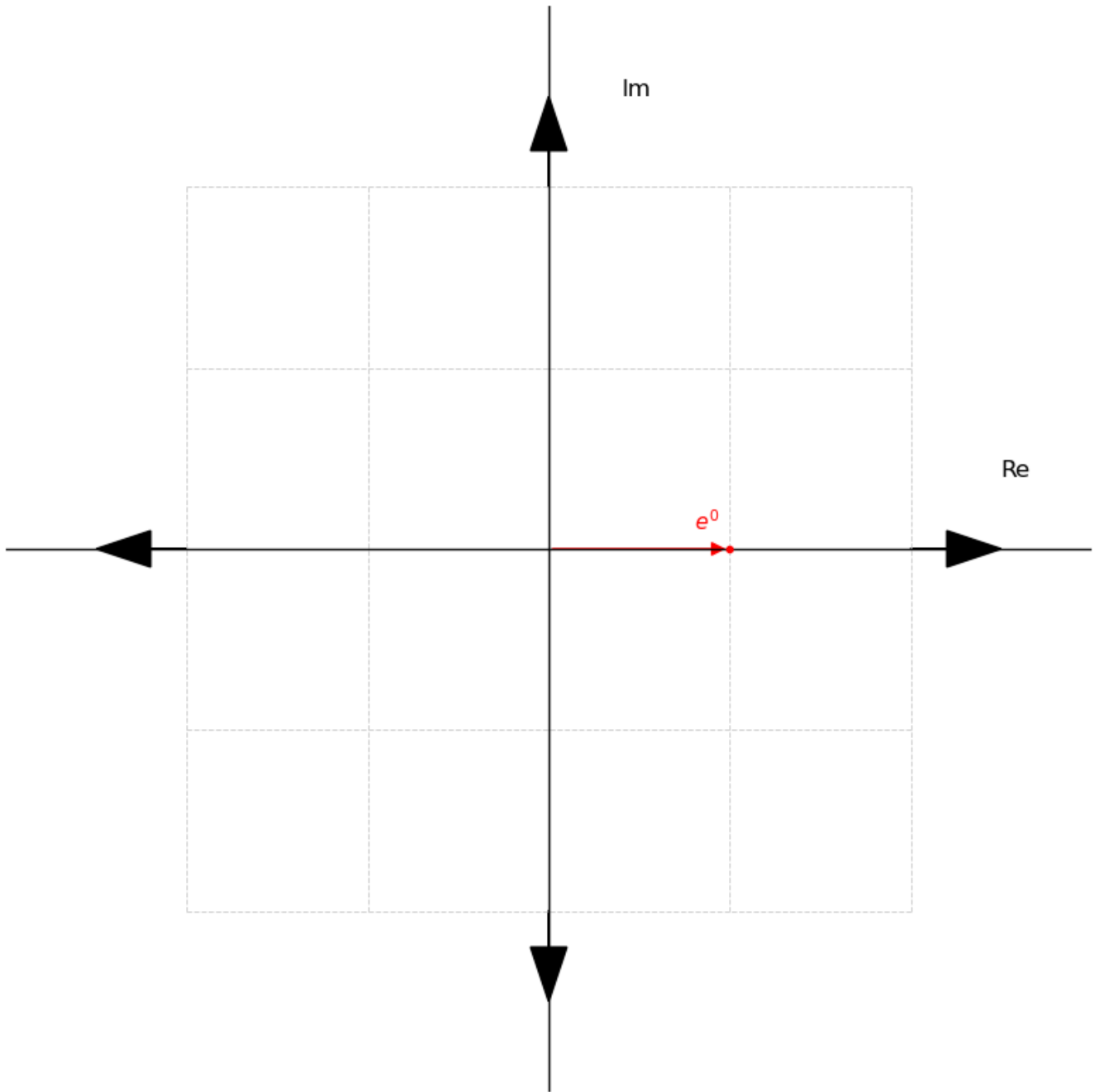


- The fourth power:

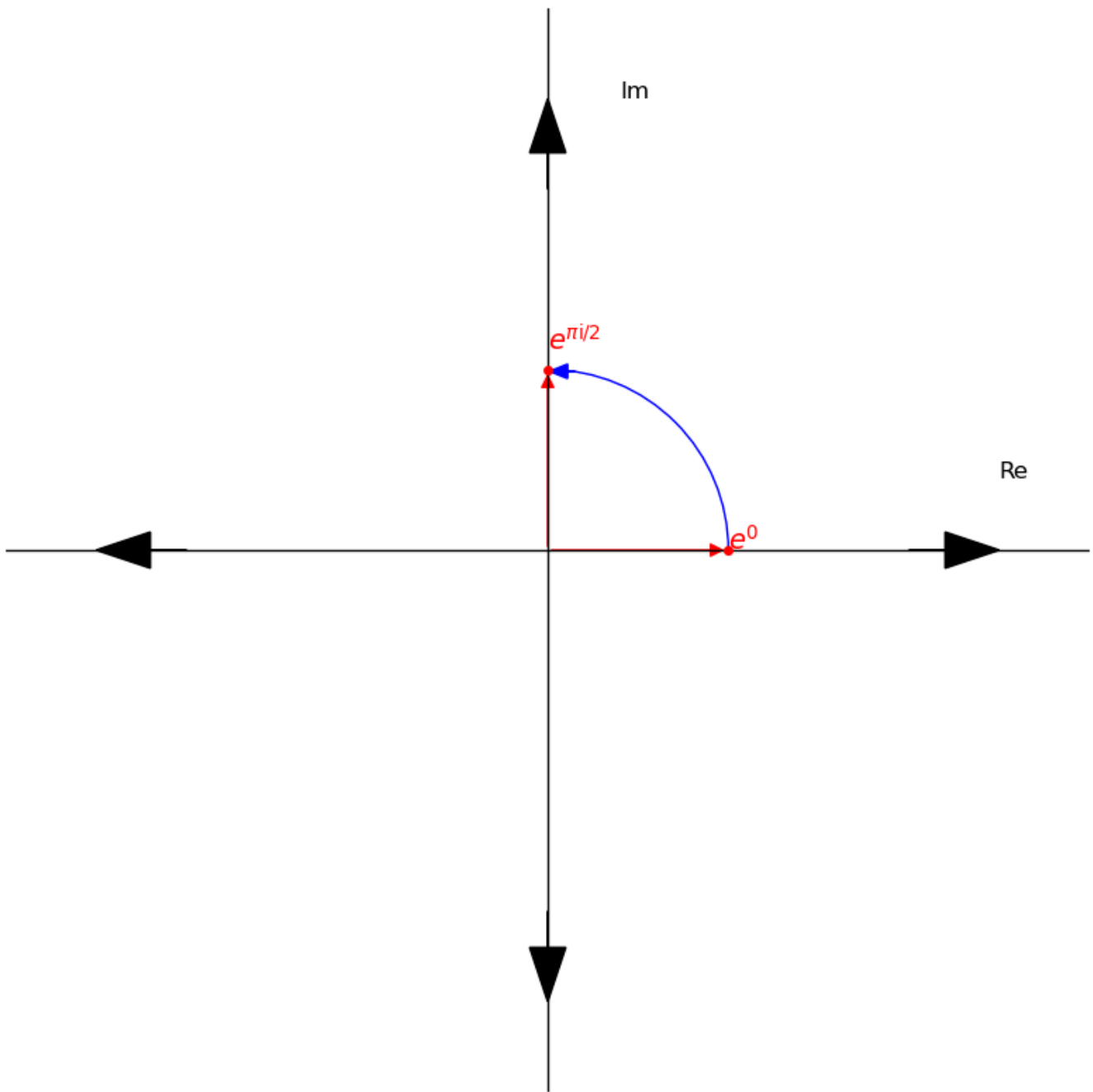


For :

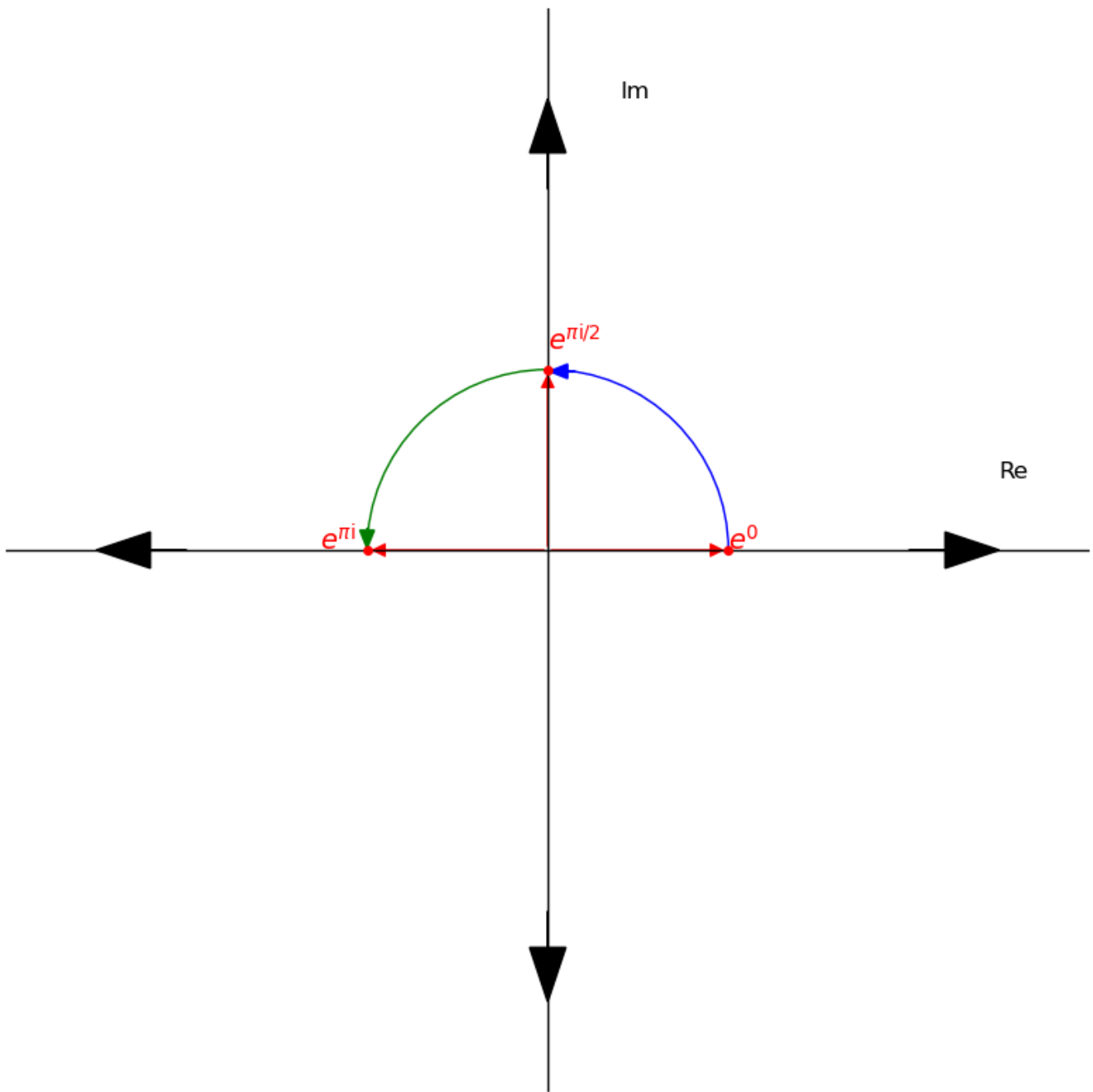
- We compute the zero power:



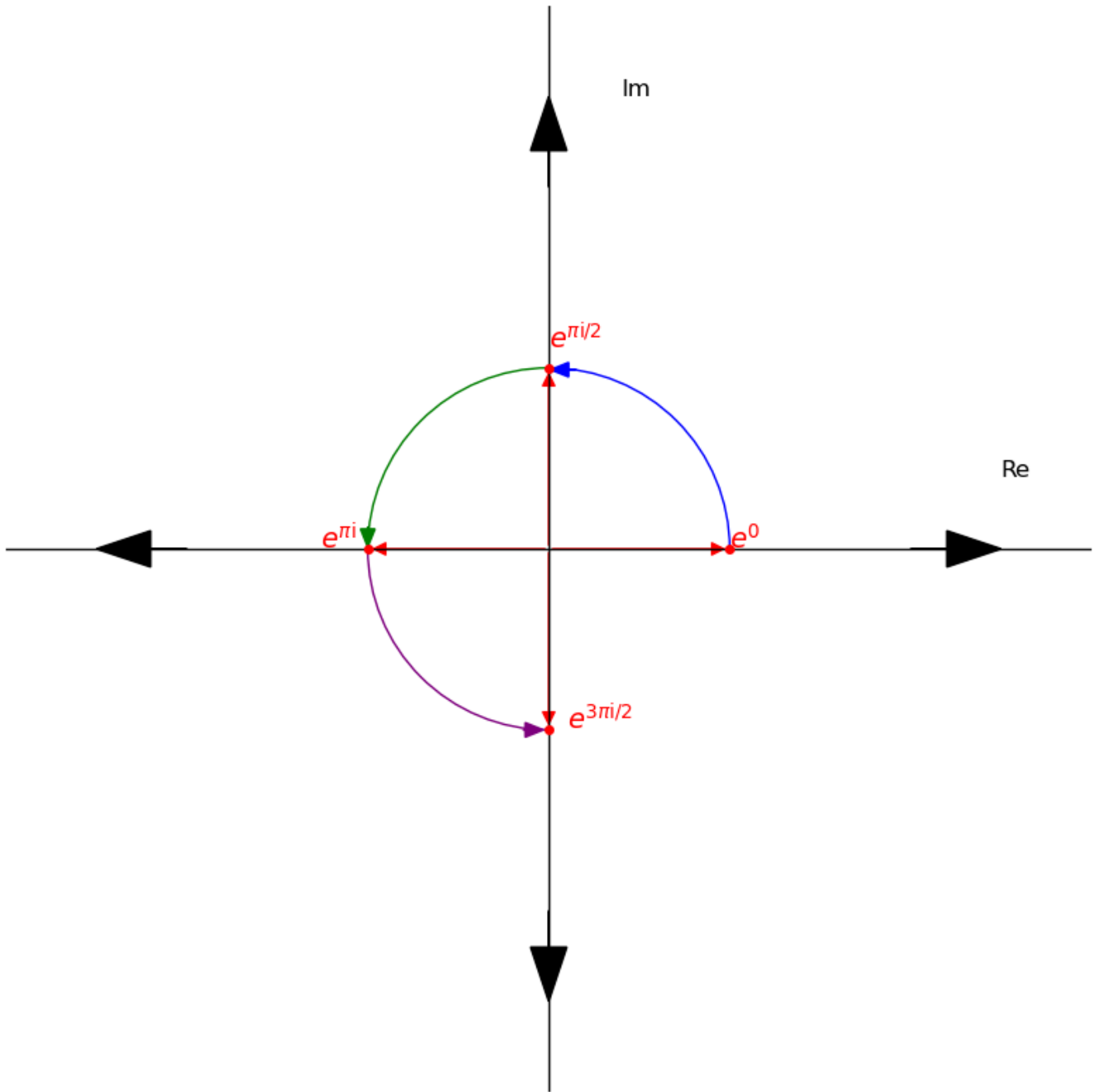
- The first power:



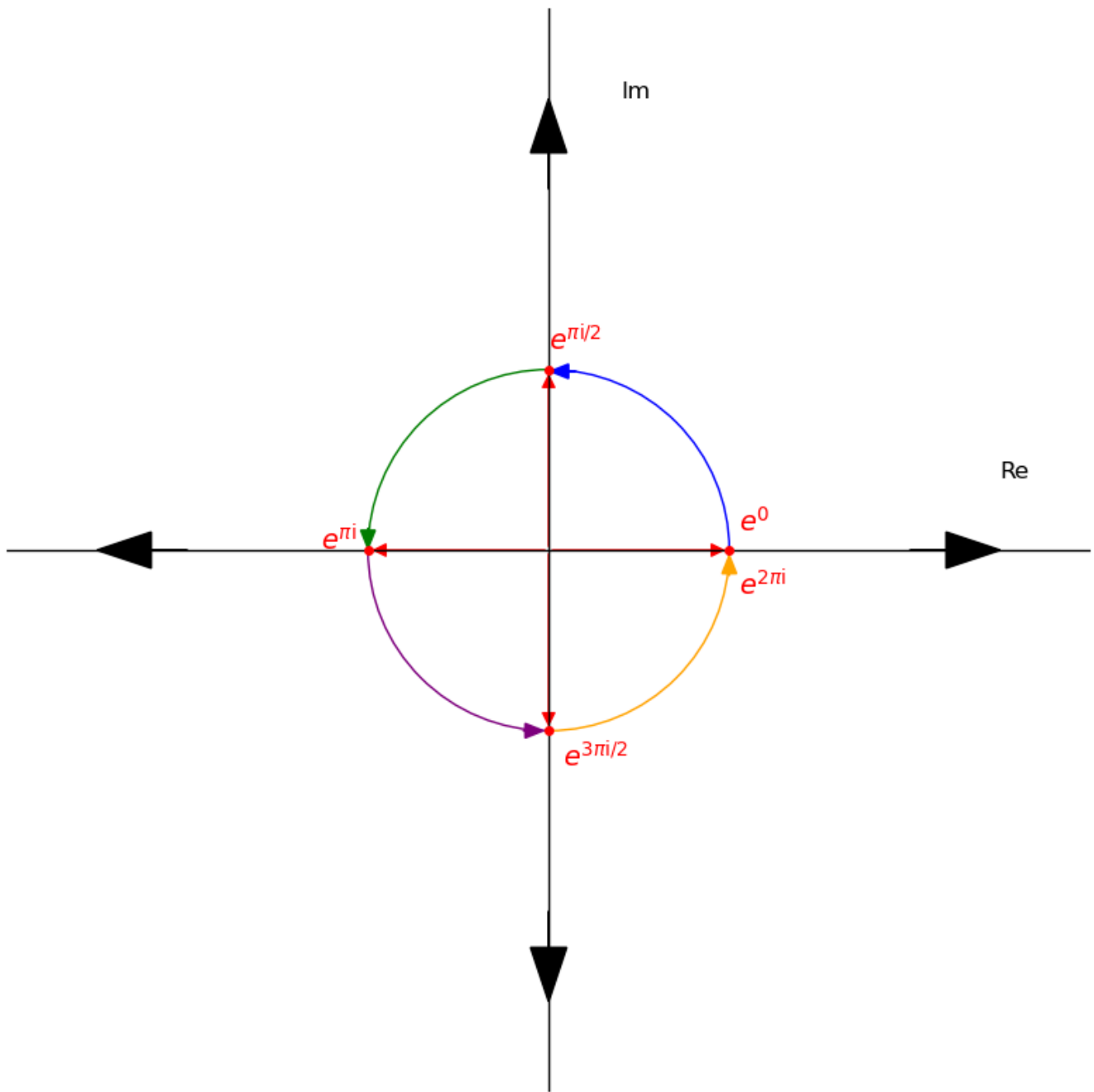
- The second power:



- The third power:

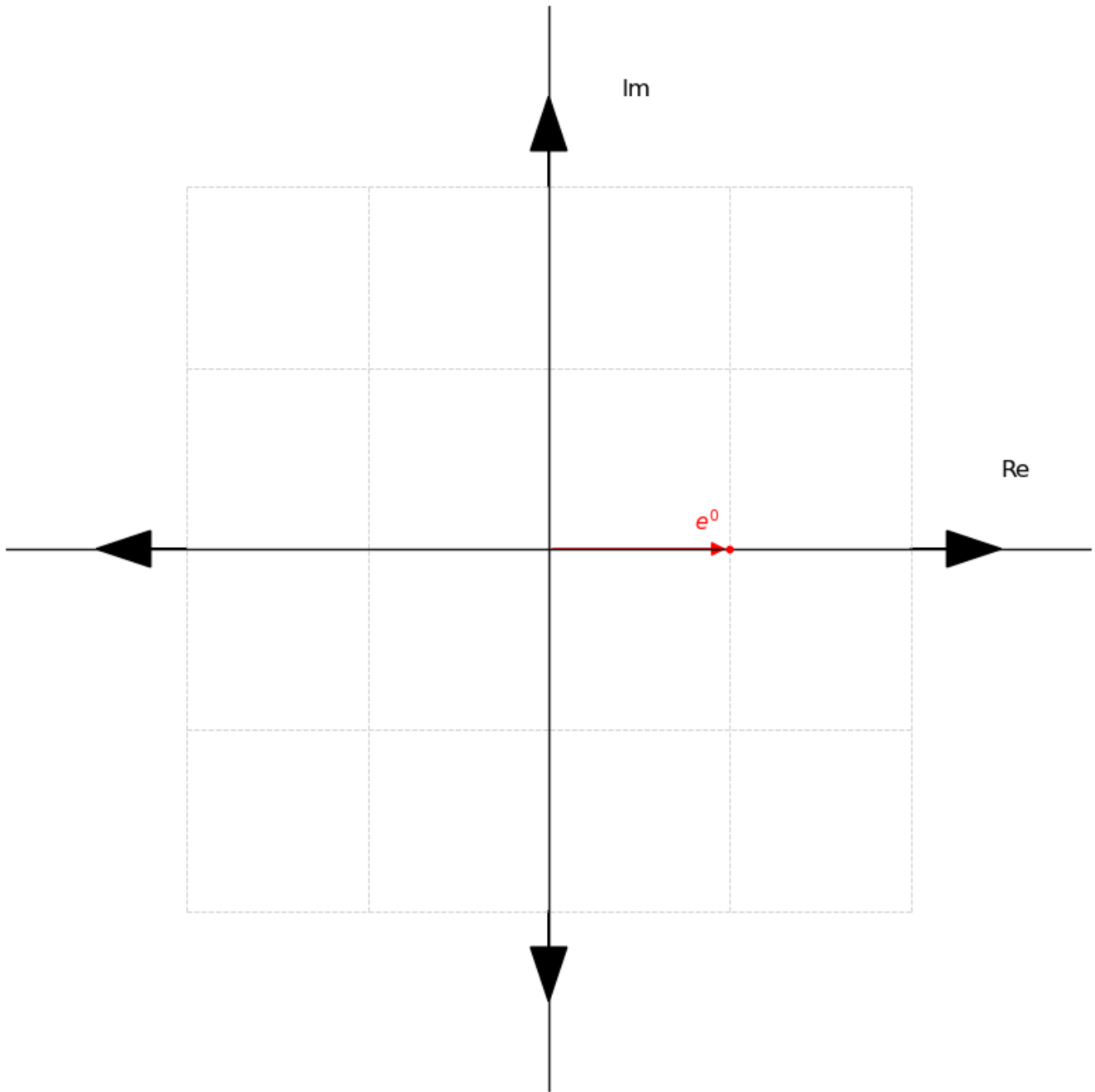


- The fourth power:

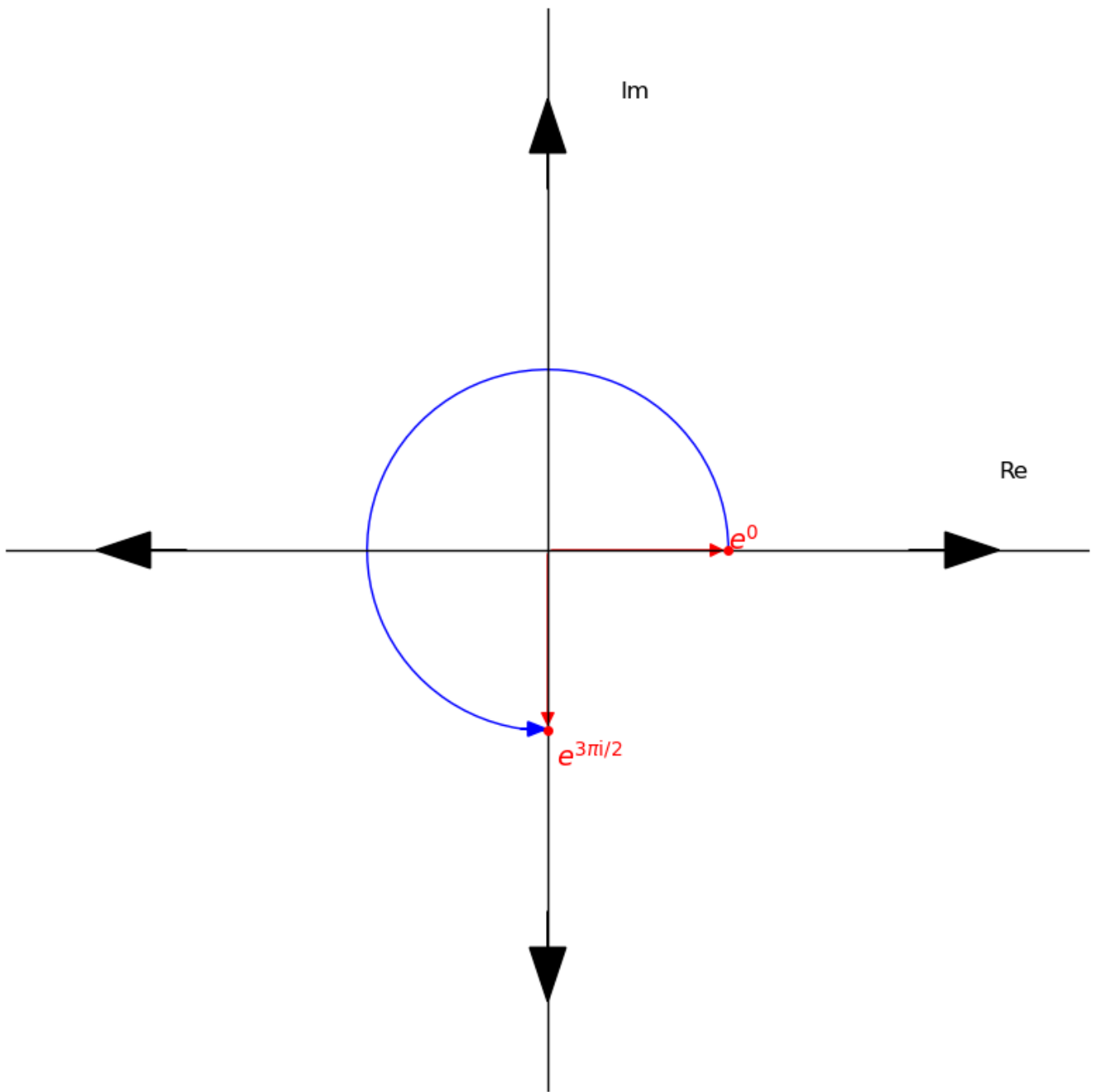


For :

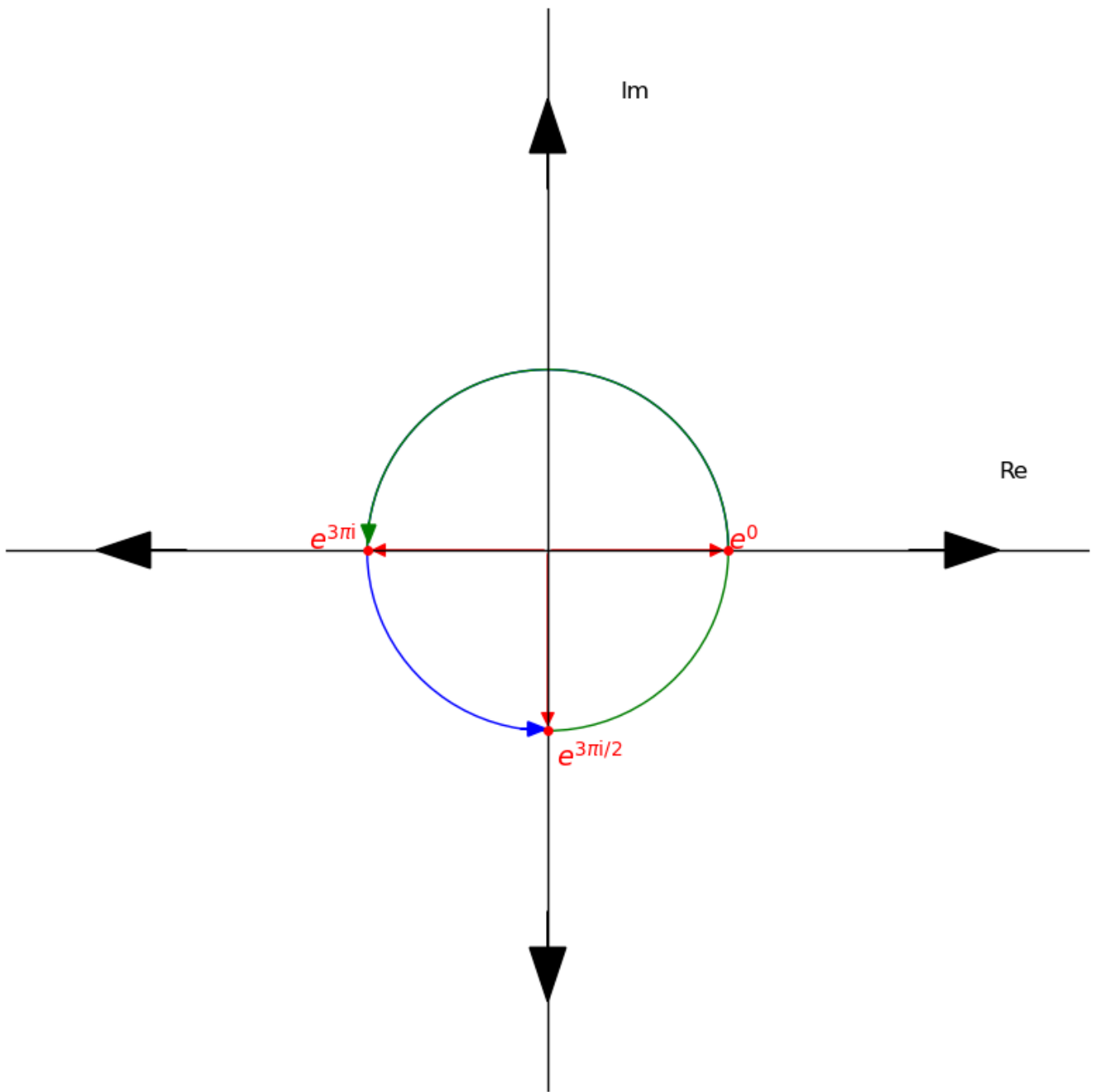
- We compute the zero power:



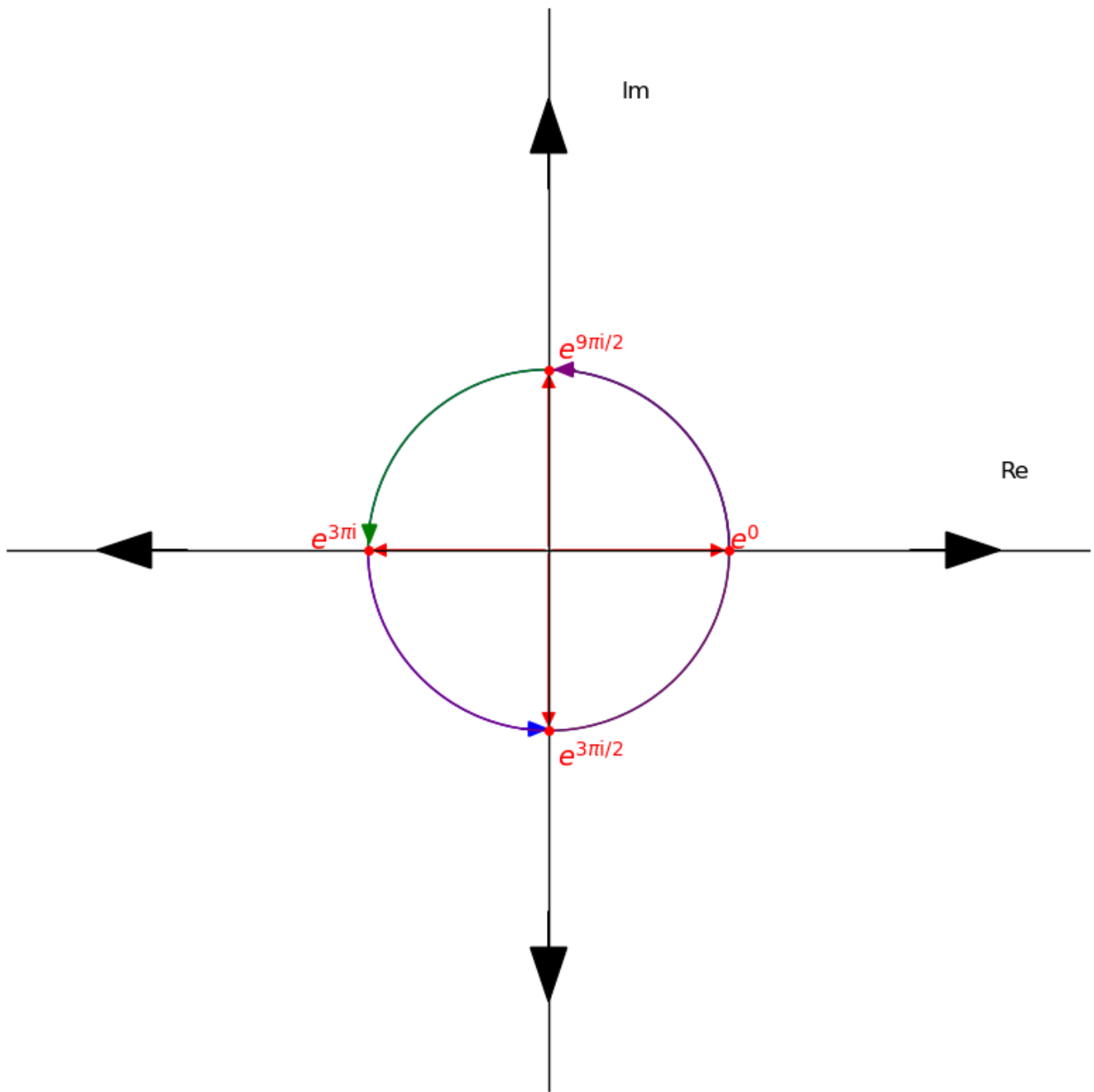
- The first power:



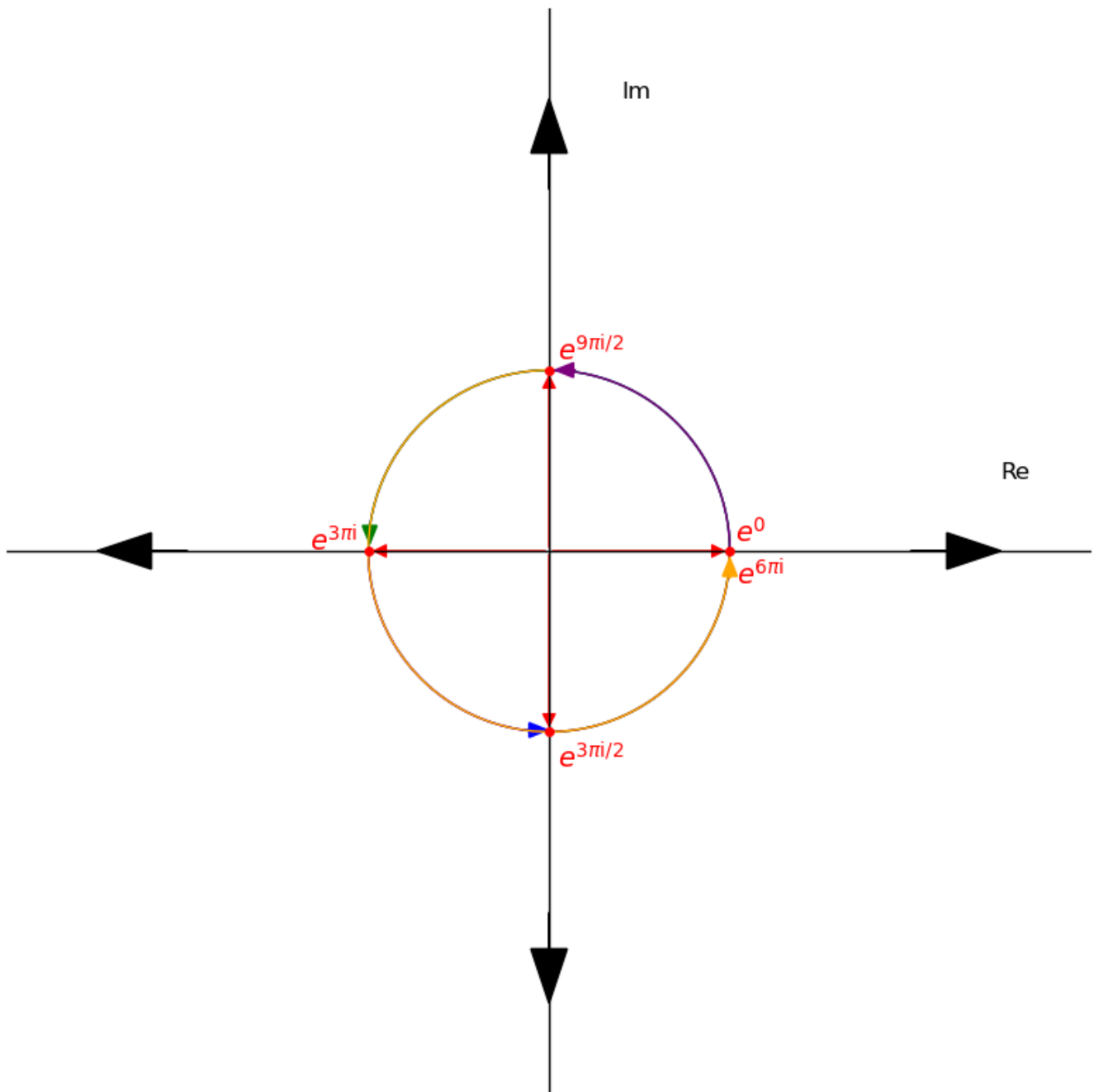
- The second power:



- The third power:



- The fourth power:



What do we notice?

- For $n=1$:
 - They are no rotations. It just remains at e^0 .
- For $n=2$:
 - It rotates between e^0 and $e^{2\pi i}$ and it's at point $e^{\pi i}$ at the powers divisible by 2 ($e^{2\pi i}$ and $e^{4\pi i}$).
- For $n=3$:
 - It rotates across all roots and meets e^0 at the fourth power.
- For $n=4$:
 - Same as $n=3$.

Finally, what are roots of unity and why does all this matter?

The n th roots of unity are simply values of z that satisfy the function $z^n = 1$ for an arbitrary value.

Having known this, notice how the roots 1 , ω , and ω^2 all satisfy function $z^3 = 1$. That is:

- $1^3 = 1$ and
- $\omega^3 = 1$ and
- $\omega^2^3 = 1$ and
- $\omega^4 = 1$ and

And, how ω^2 is equals ω^{-1} .

These are 3rd roots of unity.

But then, what about the polynomial $z^4 - 1 = 0$, does that hold for it too? Let's find out.

Given the polynomial $z^4 - 1 = 0$, let's find the roots.

- $z^4 - 1 = 0$

Setting them to zero, we have that:

- $z^4 - 1 = (z^2 - 1)(z^2 + 1)$,
- $z^2 - 1 = 0$, we already know the roots are 1 and ω^2 .

Therefore, 1 , ω and ω^2 are the 4th roots of unity. That is:

- $1^4 = 1$ and
- $\omega^4 = 1$ and
- $\omega^2^4 = 1$ and

Let's extract more insights from our argand diagrams.

Notice how in the 4th roots of unity, ω and ω^3 generates every other root in the set and in the 3rd roots of unity ω and ω^2 also generates every other other roots in the set. For example,

- For ω in the 4th roots of unity:
 - ω^2
 - $\omega^4 = 1$
 - $\omega^6 = \omega^2$
- For ω^2 in the 3rd roots of unity:
 - $\omega^4 = 1$

-
-
-

When a root generates all other roots in the set, it is called a **primitive root of unity**. This is evident in the diagrams as ω and ω^2 hits every root of unity as it rotates around the circle.

There's something that has been recurring ever since we started talking about roots of polynomials: the modulus has always been 1. This is not a coincidence.

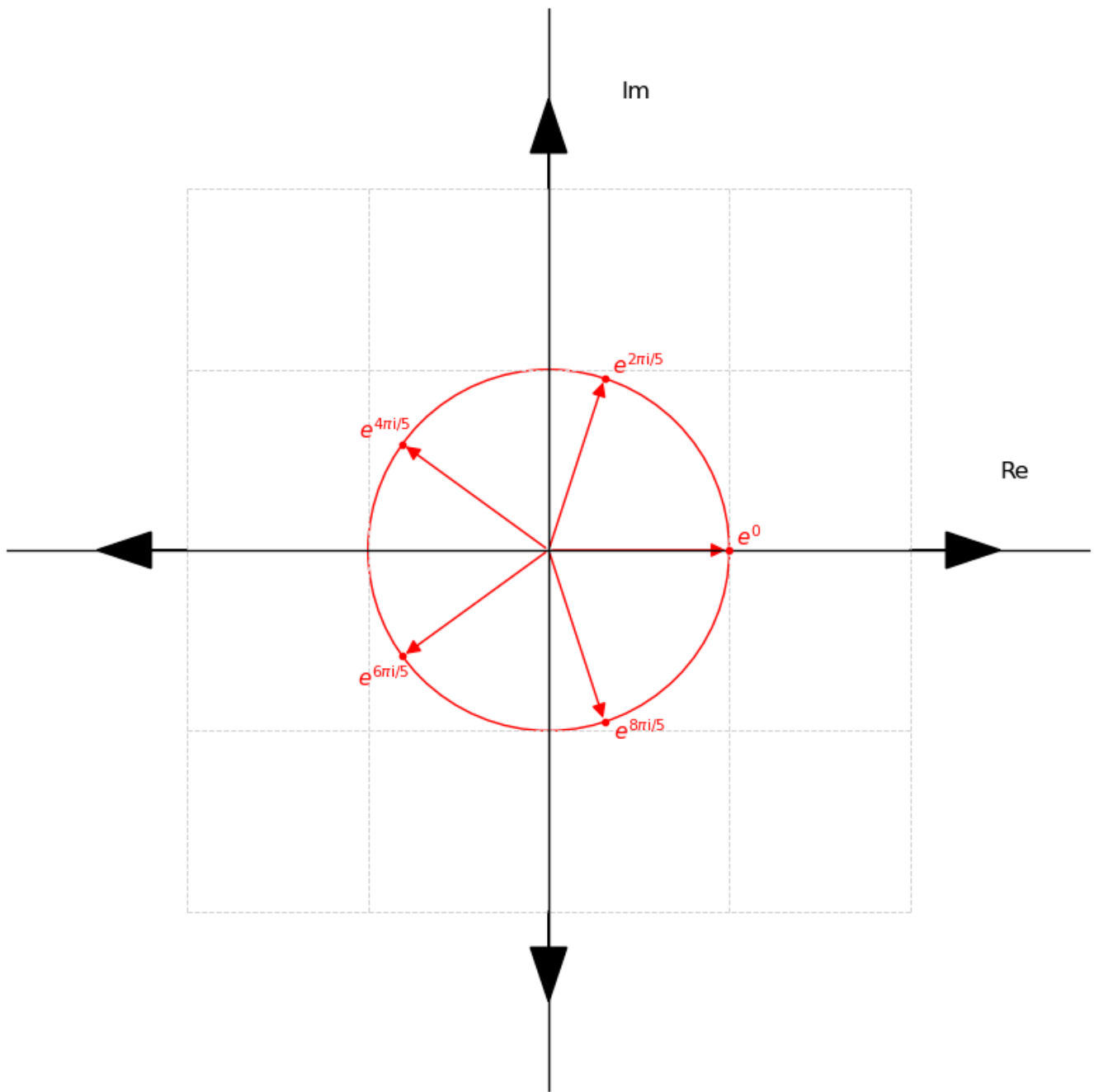
If you interpret 1 as the radius of a circle being the **unit circle**(i.e a circle of radius 1) then every root of unity is a point on the unit circle hence the name *roots of unity*.

And, there's a general formular for representing every n th root of unity:

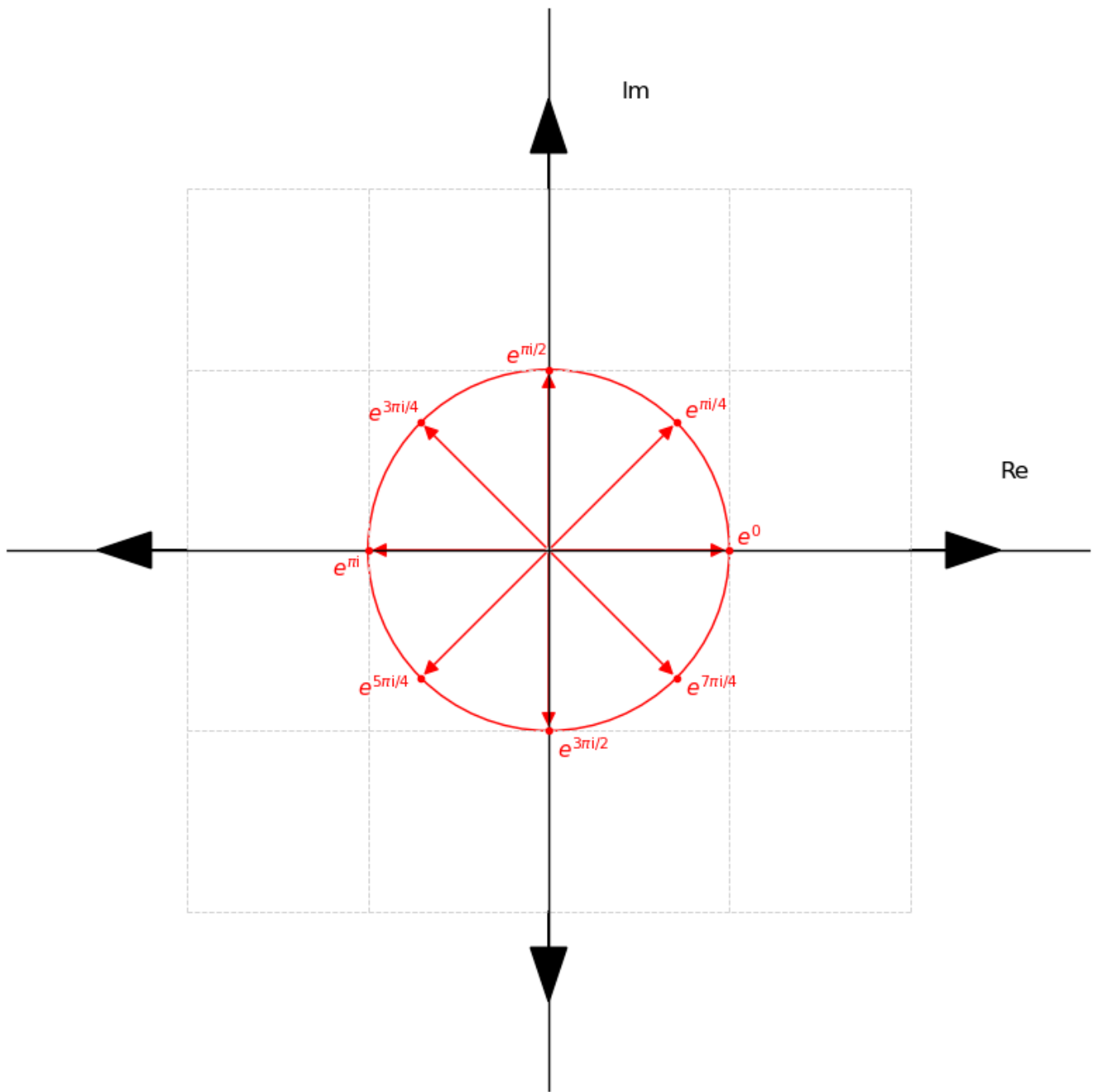
where θ is an arbitrary value and n is a positive integer.

For example:

- The n th roots of unity are



- The 5th roots of unity are



A common way of representing roots of unity is in terms of the primitive root of unity. Recall that a primitive root of unity generates all other roots of unity.

Given a primitive root of unity ω , the roots of unity are ω^k where k is an arbitrary positive number.

For example, from the 8th roots of unity, we pick the primitive root ω . That 8th roots of unity are as follows:

Another important property of roots of unity is that every root has a *complex conjugate* and that complex conjugate is the point symmetric to it on the unit circle.

The complex conjugate of a complex number $a + bi$ is $a - bi$ and that of $a - bi$ is $a + bi$. But then, when these are on the unit circle, the complex conjugate of any root is simply it's vertical reflexion.

For example, from the 8th roots of unity above, Here are the complex conjugates:

- 1 is 1
- $e^{i\pi/4}$ is $e^{-i\pi/4}$
- $e^{-i\pi/4}$ is $e^{i\pi/4}$

Here are the 8th roots of unity in rectangular form for clarification:

$$\begin{matrix} - & - & - & - & - & - & - & - \\ \hline & & & & & & & \end{matrix}$$

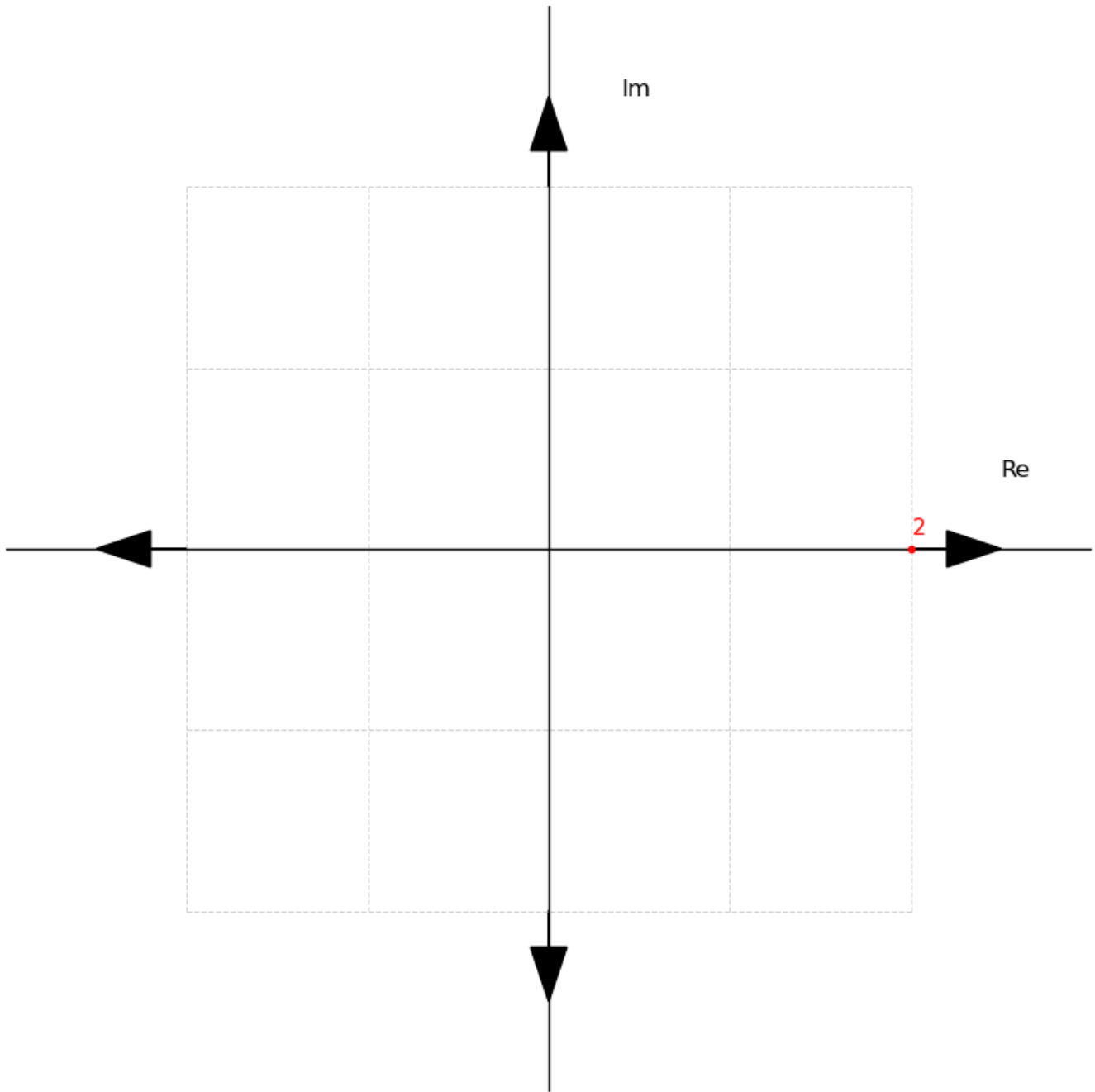
Interestingly, the complex conjugate of any root $e^{i\theta}$ is $e^{-i\theta}$. That is:

- $e^{i\theta}$ is $e^{-i\theta}$
- $e^{-i\theta}$ is $e^{i\theta}$
- 1 is 1

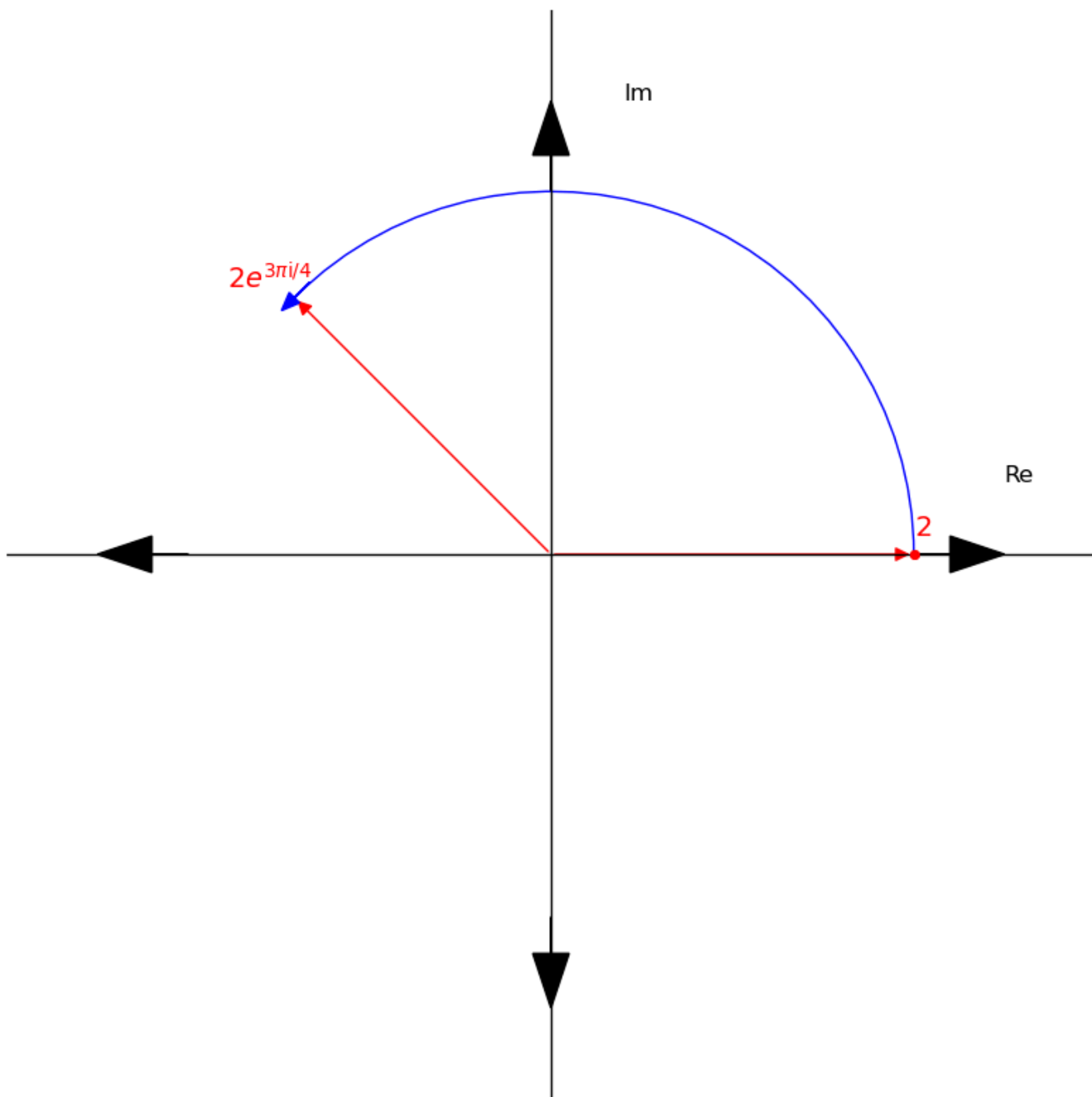
A good mental model for complex conjugates is anticlockwise vs clockwise rotations. For example $e^{i\theta}$ is the angle θ in the anticlockwise direction while $e^{-i\theta}$ is the same angle in clockwise direction.

An even better way to interpret this is *performing a rotation and undoing that rotation*.

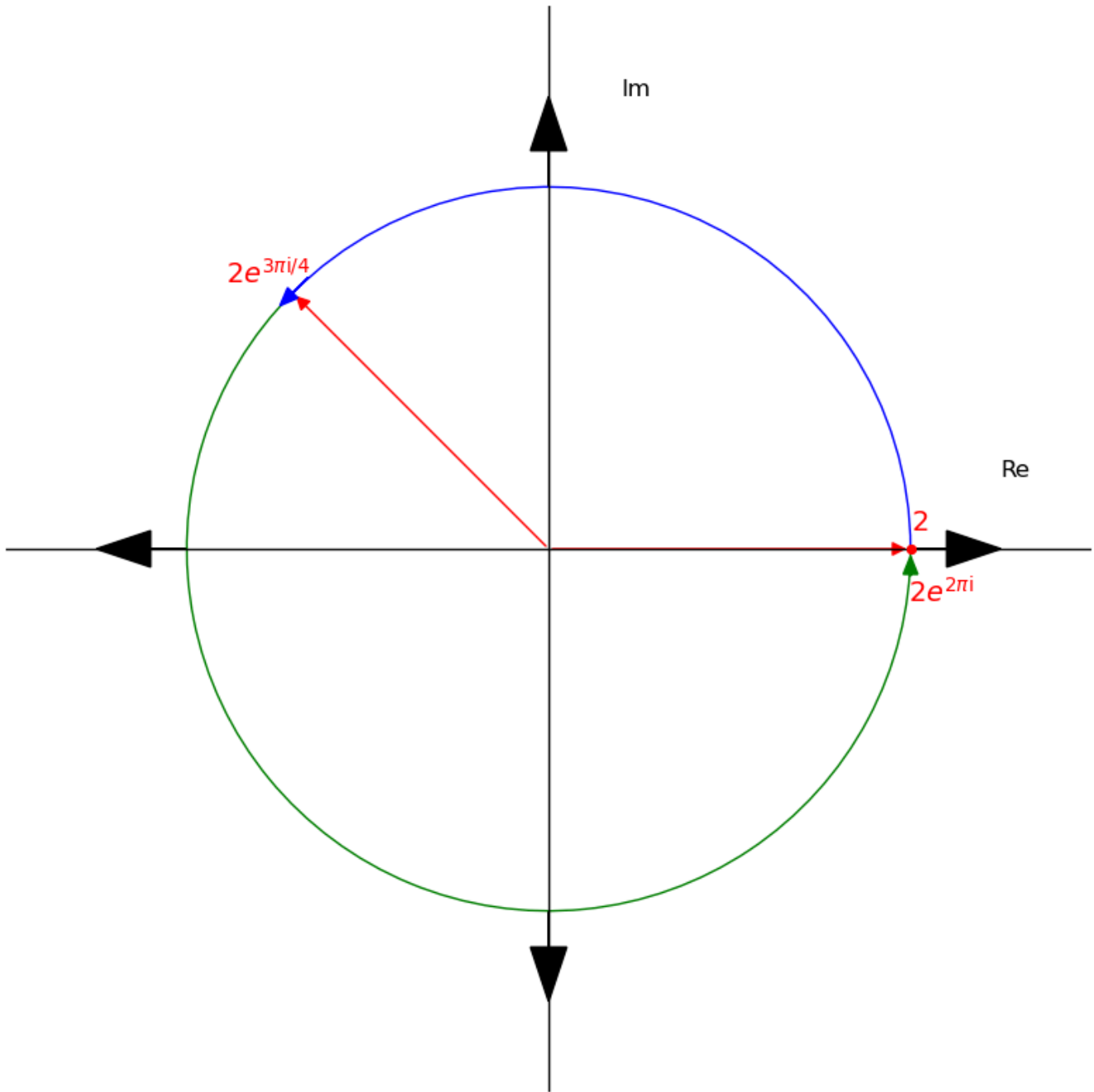
For example, in the diagram below, we have a value $e^{i\pi/4}$.



Let's multiply 2 by . We have .



Now, let's multiply $2e^{3\pi i/4}$ by the complex conjugate of $2e^{3\pi i/4}$, $2e^{-3\pi i/4}$. That is $2e^{3\pi i/4} \cdot 2e^{-3\pi i/4}$.



As you can see in the diagrams rotated by and undid that rotation taking back to .

This is a key concept in next our topic: **Discrete Fourier Transform** as we get closer to understanding FFT.

Discrete Fourier Transform (DFT)

Let's go back to where it all started: *Polynomial multiplication*.

Recall, we chose a set of values, and evaluated to get the set of points and we called it the **point representation**.

Now, let's use roots of unit as those set values; our evaluation points. Given a primitive root of unity ω , we have our roots of unit ω^k .

With this, we have the following:

Briefly, what is DFT? DFT is an algorithm that turns a signal from the time domain into the frequency domain. It is used in Signal analysis, Image Compression and a lot more.

A good way to look at DFT is an algorithm that seeks to get more information from individual values that were created out of a combination of different measurements of values. For instance, given a chord (a chord is a set of notes played together), DFT can be used to get the frequency of the individual notes that make up the chord.

Why do we care about DFT? The DFT algorithm is equivalent to polynomial evaluation using roots of unity as we shown above. It runs in $O(N^2)$ time and **FFT is an optimization of DFT and it run in $O(N \log N)$ time.**

We are going to see how DFT is used and after that we are going to see how FFT is derived from the construction of DFT.

Let's see an example. Using the N th roots of unity, let's compute $X[k]$ where $X[n]$ is the input sequence, and N is the length of the sequence.

- **Step 1:** Pick a primitive root of unity and define the the vandermonde matrix

- **Step 2:** Compute the DFT of $X[n]$ and $X^*[n]$, and $X[n]$ respectively.

The formula for computing the DFT is $X[k] = \sum_{n=0}^{N-1} x[n] e^{-j 2 \pi k n / N}$ where N is the number of samples and $e^{-j 2 \pi k n / N}$ is the N th root of unity.

This is the same as $X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}$ from earlier but updated to show that we are using the roots of unity as our evaluations points.

:

:

- Step 3: Compute the pairwise multiplication

- Step 4: Compute the inverse DFT of $X[k]$.

This takes us back to coefficient form. The formula for this is

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{j2\pi kn/N}$$

This simply means we perform the same operation as before but now we pick the positive primitive root of unity and pick k , then we divide the resulting $X[k]$ by N being 1 in this case.

This formula is different from the one we used this process earlier because of the peculiarities of complex numbers and roots of unity but it's requires less operations. For instance, we don't need to find the inverse of the matrix W_N .

—
—
—

To better understand $\mathcal{F}$, recall that $\mathcal{F}$ is $\frac{1}{\sqrt{2\pi}}$, $\mathcal{F}$ is $\frac{1}{\sqrt{2\pi}}$ and $\mathcal{F}$ is

As you can see, we have successfully performed polynomial multiplication using DFT! Finally, let's see how FFT works!

How FFT works

FFT is a recursive algorithm that exploits the structure of the DFT construction and two properties of roots of unity.

So far, we have explored DFT using matrices and matrix multiplication. But, to better understand FFT you need to view DFT in terms of the formulas $X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}$ and $x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{j2\pi kn/N}$ so let's do that.

Using same $X[k]$ and $x[n]$ from above, let's compute $X[k]$ and $x[n]$ using the formulas

For DFT:

- $X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}$:

- $x[0]$
 - $x[1]$
 - $x[2]$
 - $x[3]$

- $x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{j2\pi kn/N}$:

- $X[0]$
 - $X[1]$
 - $X[2]$
 - $X[3]$

For IDFT: $x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{j2\pi kn/N}$

- $X[0] = \sum_{n=0}^{N-1} x[n]$ —

- $X[1] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi n/N}$ —

- $X[2] = \sum_{n=0}^{N-1} x[n] e^{-j4\pi n/N}$ —

- $X[3] = \sum_{n=0}^{N-1} x[n] e^{-j6\pi n/N}$ —

Secondly, here are the two properties of roots of unity that ultimately makes FFT possible.

1. $e^{j2\pi k/N}$ is a primitive root of unity. This is gotten from the fact that $e^{j2\pi k/N}$ is a primitive root of unity if and only if k and N are coprime.
2. $e^{j2\pi k/N}$ is a primitive root of unity if and only if k and N are coprime. For example, $e^{j2\pi 1/N}$ is a primitive root from the N th of unity and $e^{j2\pi 2/N}$ is being a primitive root from the $N/2$ th roots of unity.

As you probably guessed, these only holds when N is even.

Haven set the foundation, let's look at the FFT algorithm step by step.

Starting from the DFT formula:

- **Step 1:** We are going to split the formula into even/odd indices. Setting $n = 2m$ and $n = 2m + 1$ so that:

Notice the second property at play here.

- **Step 2:** Let's label the parts:

So that:

- **Step 3:** Let's compute:

From the first property, we can update the as follows:

So, we just have to compute X_0 , X_1 and X_2 and use it for both computations, flipping the sign on the second one.

- **Step 4:** We perform same steps recursively for $N/2$ and $N/2$ until 1 .

Let's go through an example using $N = 8$ using the 8th roots of unity.

We compute:

and

To solve we just have to compute $\frac{\partial L}{\partial \mathbf{w}}$, $\frac{\partial L}{\partial \mathbf{b}}$ and $\frac{\partial L}{\partial \mathbf{c}}$.

Now, let's use it in :

In the same vein, let's look into inverse FFT.

Starting from the inverse DFT formula: —

—

—

The even and odd parts:

Then:

—

—

Let's continue from the last example. We got the DFT as
compute the inverse DFT using inverse FFT.

. Let's

—

—

and

—

—

We will compute , , and .

Now, let's use it in :

—

—

—

—

—

—

—

—

—

—

—

—

—
—
—
—
—

As you can see FFT and inverse FFT are the same algorithm with changes in the primitive root and division by N at end.

Note that this only works when N is power of 2. There are other versions of FFT that deal with other numbers but that's out of the scope right now and this is recursive algorithm so we keep perform them on the even/odd parts until N is 1.

Below is an implementation of both algorithms in python

```
def fft(x):
    N = len(x)
    if N == 1:
        return x

    wN = cmath.exp(-2j * cmath.pi / N)

    even = fft(x[0::2])
    odd = fft(x[1::2])

    X = [0] * N
    w = 1
    for k in range(N // 2):
        t = w * odd[k]
        X[k] = even[k] + t
        X[k + N // 2] = even[k] - t
        w *= wN

    return X

def ifft(X):
    N = len(X)
    if N == 1:
        return X

    wN = cmath.exp(2j * cmath.pi / N)
```

```
even = ifft(X[0::2])
odd  = ifft(X[1::2])
```

```
x = [0] * N
w = 1
for k in range(N // 2):
    t = w * odd[k]
    x[k] = even[k] + t
    x[k + N // 2] = even[k] - t
    w *= wN
```

```
return [v / 2 for v in x]  # divide by 2 at each level → 1/N total
```

```
x = [1, 2, 3, 4]
X = fft(x)
xr = ifft(X)
```

```
print(X)
print(xr)
```

Lastly, let's briefly look at Number Theoretic Transform and how it relates to FFT.

Number Theoretic Transform (NTT)